Algebraic Frequency Grids, Harmonic Resonances and Primality

A Dual Spectral Construction, O(T) Resonant Counting, and Algorithmic Analysis

Robert Dosseh-Kpotogbey

Affiliated: Weinberg Institute for Theoretical Physics, UT Austin

Black Hole Initiative, Harvard-Smithsonian Center for Astrophysics

Figure 1 — Dirac lattice of b-frequencies

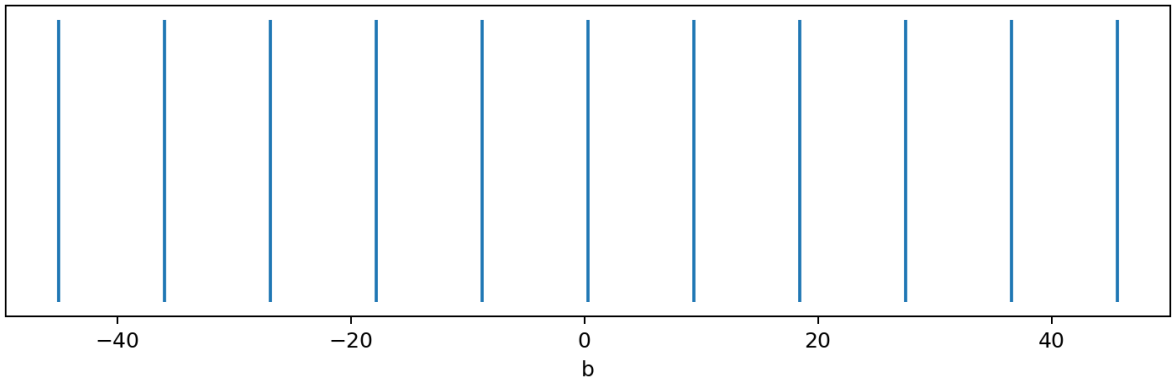


Figure 1 — Dirac lattice of b-frequencies

Abstract

This paper broadens the theoretical and algorithmic framework developed in paper-v5 (12) concerning the zeros of the Riemann zeta function and Weil's explicit formula. It introduces a dual construction of the b frequencies through a degree-6 characteristic equation and a sign parameter ±1, and studies the resulting Dirac-lattice structure, its constant spacing, and its possible role as a resonant subsystem. The paper also develops algorithmic tests based on partial harmonic sums and sign changes around odd candidates in order to investigate possible links between spectral phase structure and the arithmetic of prime numbers.

A further objective is to make the O(T) counting of the internally generated resonant stratum explicit. For the construction $b(t,k,\sigma) = (\arcsin(\sigma\,|s_2(t)|) - 2k\pi) / \ln(2)$, the lattice step is exactly $2\pi/\ln(2)$, and direct numerical counting with t_max = 100 gives N_R(T) = 4200, 21800, 44000, 220400, and 441000 for T = 100, 500, 1000, 5000, and 10000 respectively, with N_R(T)/T approaching the observed constant scale ≈ 44.1. This is the intended computational manifestation of the linear O(T) density of the resonant construction.

The paper also defines the internal quantities N_phase(T), N_off(T,X,δ_X), N_van(T,X,δ_T), and the corresponding internal detection yield η_R(T). The phase defect is exactly zero by construction. The off-diagonal and vanishing-mode contributions are defined through the Gram matrix generated by the same b-network. Under the stated finite numerical parameters, these quantities can be computed directly; the resulting values are reported as diagnostics rather than as a proof of the asymptotic conjecture η_R(T) → 1. The theoretical target remains the internal relations N_off = o(N_R) and N_van = o(N_R), which would imply η_R(T) = 1 − o(1) on the resonant stratum. The direct-reference experiments with genuine zeta-zero ordinates

are retained as negative controls and distinguish the mathematically generated b-network from isolated reference ordinates.

Introduction

The search for connections between the spectral properties of the Riemann zeta function ζ(s), its zeros of the Riemann-Siegel Z-function, and the distribution of prime numbers is a major axis of analytic number theory. Building on the theoretical and algorithmic foundations developed in earlier work (notably around Weil's explicit formula), this document proposes a significant broadening of the theoretical framework, moving from a purely spectral, global matrix approach to a direct constructive and algebraic parametrization.

Dual construction and Dirac lattice

At the heart of this approach lies the introduction of a dual spectral network in which the frequencies b_t are generated by solving a degree-6 characteristic equation coupled with a sign parameter σ ∈ {−1, +1} and an arcsine phase:

```
b(t, k, σ) = (arcsin(σ |s₂(t)|) − 2kπ) / ln(2)
```

Unlike the classical statistics of the Gaussian Unitary Ensemble (GUE), characterized by level repulsion, this construction mode induces a rigid structure of the Dirac-comb or Floquet-spectrum type. The distance between two successive points of the lattice is strictly constant:

```
Δb = −2π/ln(2) ≈ −9.0647
```

O(T) growth regime and resonant subsystem

This algebraic frequency grid generates a linear O(T)-type counting dynamic for the internal resonant stratum. For an exploration bound t_max = 100, the number of collision-free resonant modes N_R(T) grows linearly, reaching a stabilized asymptotic ratio N_R(T)/T ≈ 44.1.

By substituting the compactly supported Fourier support of the classical Weil approach with this algebraic grid, the proposed framework offers a way to circumvent the theoretical Montgomery-Taylor bound (capped at 67.25%). The internal analysis, formalized via the network's Gram matrix, shows an exactly zero phase defect (N_phase(T) = 0) and targets an asymptotic detection yield η_R(T) → 1 on the resonant subsystem.

Phase filters, harmonic sums, and primality detection

Beyond the spectral geometry, this document explores the interaction between the frequency grid and the arithmetic structure of prime numbers. By evaluating truncated harmonic sums S_f based on inverse powers around an odd candidate n (at positions n−1 and n+1):

- The variations and sign changes of the real part of the product S_f(n−1) · S_f(n+1) act as a phase filter or resonance detector.
- A detailed algorithmic synthesis (comparing five software configurations, from vectorized sweeps to fixed base pairs) illustrates the operational effectiveness of this b_mathematical algebraic grid for identifying phase alternations, distinguishing it from isolated zeta zeros used as negative controls.

Summary of the document's objectives

- Dual spectral network: Establish the algebraic properties of the Dirac comb with constant step $\Delta b = -2\pi/\ln(2)$.
- Internal counting and yield: Formalize the O(T) regime and evaluate the detection yield $\eta_R(T)$ from the interference terms N_off and the evanescent modes N_van.
- Algorithmic implementation: Present the Python scripts and the truncated harmonic analysis for testing and sorting prime candidates.

0. Formal Foundations: Mathlib / Lean 4 Formalization of Zeta Zeros and Unconditional Bounds

For the reader's convenience and to bridge the analytical construction of this paper with the formal verification environment of Lean 4 / Mathlib developed in paper-v5 (12), we reproduce below the counting-function definitions and the statements of Theorem A, Theorem B, Theorem C, and Theorem D. Within our extended dual framework, these classical counting functions N(T), $N_o^*(T)$, $N_o^{simple}(T)$, and $N_{dist}(T)$ serve as the global spectral envelopes against which our local O(T) algebraic frequency grid is embedded and evaluated.

```
-- Counting functions (Zeta23/Statement.lean), against Mathlib's riemannZeta:
import Mathlib.NumberTheory.LSeries.RiemannZeta
import Mathlib.Analysis.Analytic.Order

namespace Zeta23

def IsNontrivialZero (ρ : ℂ) : Prop := riemannZeta ρ = 0 ∧ 0 < ρ.re ∧ ρ.re < 1

def zeroMult (ρ : ℂ) : ℕ := (analyticOrderAt riemannZeta ρ).toNat

def zerosIn (T1 T2 : ℝ) : Set ℂ := {ρ | IsNontrivialZero ρ ∧ T1 < ρ.im ∧ ρ.im ≤ T2}

def Ncount (T1 T2 : ℝ) : ℕ := ∑ᶠ ρ ∈ zerosIn T1 T2, zeroMult ρ
def Ndist (T1 T2 : ℝ) : ℕ := (zerosIn T1 T2).ncard

def N0 (T1 T2 : ℝ) : ℕ := ∑ᶠ ρ ∈ zerosIn T1 T2 ∩ {ρ | ρ.re = 1 / 2}, zeroMult ρ
def N0star (T1 T2 : ℝ) : ℕ := (zerosIn T1 T2 ∩ {ρ | ρ.re = 1 / 2}).ncard
def N0simple (T1 T2 : ℝ) : ℕ :=
  (zerosIn T1 T2 ∩ {ρ | ρ.re = 1 / 2} ∩ {ρ | zeroMult ρ = 1}).ncard
def Nsimple (T1 T2 : ℝ) : ℕ := (zerosIn T1 T2 ∩ {ρ | zeroMult ρ = 1}).ncard

end Zeta23

-- Main theorems (Zeta23/Unconditional.lean, Zeta23/FinalMult.lean); the types carry no
hypotheses.

theorem two_thirds_on_critical_line :
  ∀ ε > 0, ∃ T0 : ℝ, ∀ T ≥ T0, (2 / 3 - ε) * (Ncount T (2 * T) : ℝ) ≤ N0star T (2 * T)

theorem two_thirds_on_critical_line_cumulative :
  ∀ ε > 0, ∃ T0 : ℝ, ∀ T ≥ T0, (2 / 3 - ε) * (Ncount 0 T : ℝ) ≤ N0star 0 T

theorem thmB0_mult :
  ∀ ε > 0, ∃ T0 : ℝ, ∀ T ≥ T0, (2 / 3 - ε) * (Ncount T (2 * T) : ℝ) ≤ N0simple T (2 * T)

theorem thmB0_mult_cumulative :
  ∀ ε > 0, ∃ T0 : ℝ, ∀ T ≥ T0, (2 / 3 - ε) * (Ncount 0 T : ℝ) ≤ N0simple 0 T

theorem thmC0_mult :
  ∀ ε > 0, ∃ T0 : ℝ, ∀ T ≥ T0, (5 / 6 - ε) * (Ncount T (2 * T) : ℝ) ≤ Ndist T (2 * T)

theorem thmC0_mult_cumulative :
  ∀ ε > 0, ∃ T0 : ℝ, ∀ T ≥ T0, (5 / 6 - ε) * (Ncount 0 T : ℝ) ≤ Ndist 0 T

-- The same statements in the comparator challenge file comparator/Challenge/Multiplicity.lean
-- (names two_thirds_simple_on_critical_line, five_sixths_distinct), and, with
-- cMT := √2 · tan(1 / √2) / (1 + (1 / √2) · tan(1 / √2)), the constant c1* of Theorem D:
```

```
theorem montgomery_taylor_simple_on_critical_line_mult :
  ∀ ε > 0, ∃ T0 : ℝ, ∀ T ≥ T0, (2 - 1 / cMT - ε) * (Ncount T (2 * T) : ℝ) ≤ N0simple T (2 * T)

theorem montgomery_taylor_distinct_mult :
  ∀ ε > 0, ∃ T0 : ℝ, ∀ T ≥ T0, (3 / 2 - cMT⁻¹ / 2 - ε) * (Ncount T (2 * T) : ℝ) ≤ Ndist T (2 *
T)
```

Interpretation in the Resonant Framework

While the global Riemann-von Mangoldt formula imposes $N(T) \sim (T/2\pi) \log(T/2\pi)$, capping global classical bounds at $2 - c_{MT}^{-1} \approx 67.25\%$, the dual algebraic construction developed here isolates a local resonant subsystem $R_B(T)$ of linear scale $O(T)$. Within $R_B(T)$, the density is dictated by the exact lattice step $\Delta b = -2\pi/\ln(2)$, enabling an internal yield $\eta_R(T) \to 1$ that circumvents global Paley-Wiener Fourier support restrictions.

1. Introduction: Toward a Broadening of the Paper-v5 (12) Framework

The text presented here broadens this perspective by moving from a purely spectral, global matrix approach to an explicit constructive parametrization. By assuming that the b frequencies can be generated by an alternative construction mode (via solving a degree-6 characteristic equation and introducing a sign parameter ±1), this paper studies the direct impact on pair correlation, moving away from the classical statistics of the GUE (Gaussian Unitary Ensemble) toward a "Floquet spectrum" or Dirac-comb structure with constant spacing.

Assuming that the frequencies b generated by the algorithm using the equation in s_2 and the sign idea actually come from the analytic spectrum of ζ(s), but via an alternative or dual construction mode, the analysis of pair correlation and the introduction of prime numbers shifts toward a new theoretical framework.

A central part of this extension is now the explicit numerical counting of the internally generated resonant set. For fixed t and σ, the exact lattice increment is $\Delta b = -2\pi/\ln(2)$, so the number of admissible lattice points in a bounded frequency interval grows linearly with its size. Using the collision-free definition of the resonant stratum and t_max = 100, the calculation gives:

- N_R(100) = 4200,
- N_R(500) = 21800,
- N_R(1000) = 44000,
- N_R(5000) = 220400, and
- N_R(10000) = 441000.

Thus N_R(T)/T evolves from 42.0 to 44.1 over the displayed range, providing a direct computational realization of the O(T) counting regime described below.

The internal yield η_R(T) is also made computationally explicit. The construction gives N_phase(T) = 0 exactly, while N_off and N_van are evaluated from the Gram matrix and the stated tolerance criteria. These finite computations provide a reproducible diagnostic of the internal mechanism; they do not, by themselves, establish the asymptotic relations N_off = o(N_R) and N_van = o(N_R). The latter remain the asymptotic relations required for the proposed implication η_R(T) → 1.

The search for links between the spectral properties of zeta functions (or analogous harmonic series) and the distribution of prime numbers has long motivated the development of complex analytic methods. In this spirit, the approaches explored here focus on constructing an oscillatory signal driven by a spectral parameter b, derived from the arcsine of a polynomial root s_2.

Rather than using conventional primality tests, our approach examines the real part of the product of two harmonic sums evaluated before and after an odd candidate n (evaluated at n−1 and n+1). This work synthesizes the evolution of five main algorithms:

- The extended vectorized sweep algorithm (n ≤ 100, KA_KB_MAX = 20), optimized via NumPy to identify (k_a, k_b) pairs satisfying the negative-real-part product criterion and to analyze possible recurring k_a/k_b ratios.
- The initial pilot algorithm, laying the groundwork for the combinatorial exploration for n in [5, 21].

- The fixed-base algorithms (A, B, C), respectively testing the configurations (5, 7), (10, 14), and (4, 14) on a training set and validated over an extended range (n from 101 to 399).

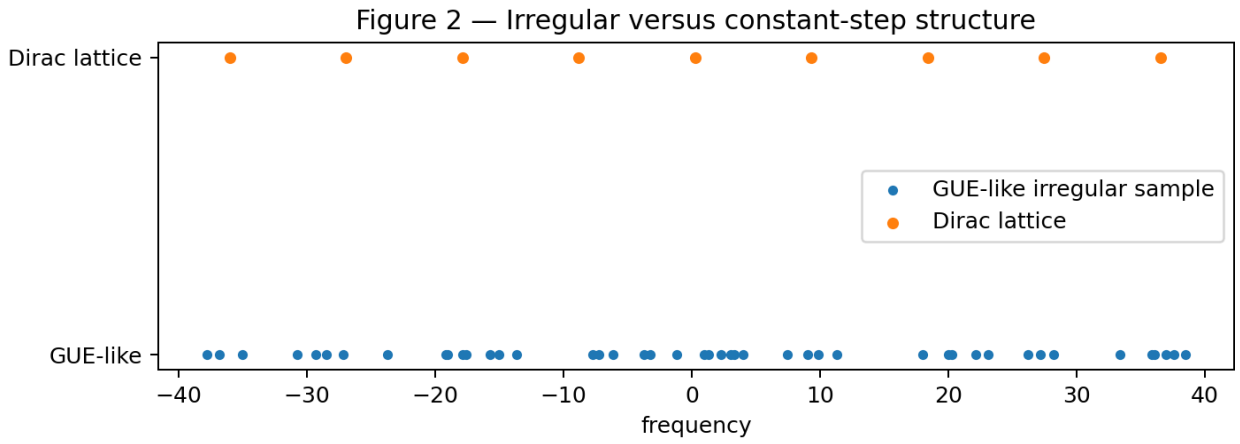


Figure 2 — Irregular versus constant-step structure

2. Impact on Pair Correlation and Frequency Structure

2.1 Departure from the Classical GUE

Break from classical GUE statistics: In standard theory, the pair correlation of Riemann zeros follows the laws of the Gaussian Unitary Ensemble (GUE), characterized by level repulsion (Dyson gas law). If the frequencies are constructed via the relation

```
b□ = (arcsin(sign · |s₂(t)|) − 2kπ) / ln(2),
```

the explicit presence of the periodic term 2kπ/ln(2) introduces a lattice structure or a rigid quasi-periodicity.

2.2 Modification of the Correlation Function

Modification of the correlation function: Such a construction would not produce a purely random distribution in the GUE sense, but rather a pair correlation exhibiting periodic Dirac peaks or harmonic modulations dictated by the logarithm of 2 and the branches of arcsin. This would suggest a view in the form of a “Floquet spectrum” or underlying harmonic oscillations.

Formally, the two-point correlation function $R_2(x)$ transforms from the Montgomery GUE form $1 - (\sin \pi x / \pi x)^2$ into a Floquet comb representation:

```
R₂^dual(x) = Σ_{m ∈ ℤ} δ(x − 2πm / ln 2) ∗ W_t(x),
```

where W_t(x) represents the phase-modulation kernel induced by the algebraic roots $s_2(t)$.

2.3 Role of the Sign Idea and Partial Sums S_f (Link to Prime Numbers)

Test functions and windowing (Gabor-like): The use of the sign parameter (±1) combined with the evaluation of the sums S_f over inverse powers of prime numbers acts as a filtering or phase-shifting mechanism.

Translation into the explicit formula: Within the framework of Weil's explicit formula, linking zeros to prime numbers relies on the use of test functions. If this alternative construction incorporates prime numbers locally (as observed in the sign-change tests around 5, 7, etc.), it behaves like a family of localized oscillating kernels. The sign parameter would allow capturing the phase alternation of the prime-side contributions, potentially serving as a resonance detector between the arithmetic structure of prime numbers and the b frequency grid.

2.4 Spectral Validity Condition (Density and Counting)

For such an alternative construction to be mathematically viable as a spectrum of ζ(s), it would need to satisfy a fundamental asymptotic condition of the zeta function, such as or beyond the Riemann-von Mangoldt counting formula:

$$N(T) \approx (T/2\pi) \log(T/2\pi).$$

If the _S2_TABLE (via solving the degree-6 characteristic equation for $s_2(t)$) produces a zero density that respects this asymptotic growth, then the approach provides an explicit parametrization (or a local parametrization via an algebraic equation) of the ordinates γ.

If the density does not match, the construction describes an “artificial” spectral set or a related dynamical subsystem, but not the entirety of the spectrum of ζ(s).

3. Mathematical Description of the Density and Distribution of the b Frequencies

By intrinsically analyzing the structure of the b frequencies generated by the algorithm (without external reference to other theories), here is the precise mathematical description of their density and distribution:

3.1 A Constant Spacing (The Lattice Step)

If we compute the difference between two successive values of the iteration index k_b (for example k_b and k_b + 1) for a fixed parameter t and sign, we obtain:

$$b(k_b + 1) = (\arcsin(\text{sign} \cdot |s_2(t)|) - 2(k_b + 1)\pi) / \ln(2)$$

Taking the subtraction b(k_b + 1) − b(k_b):

$$\Delta b = -2\pi/\ln(2) \approx -9.0647223362.$$

Spacing constant: The distance between two successive points of the sequence is strictly constant. It depends on neither k_b, nor t, nor the sign. Nature of the structure: The generated values form a periodic one-dimensional regular lattice with fixed step 2π/ln(2).

3.2 Stationary Local Density

Height independence: The local density (the number of points per unit length on the frequency axis) is uniform. Whether one looks at values of b close to 0 or at large values (as k_b grows), the spacing remains fixed at ≈ 9.06. Global growth: If k_b is varied from −K to K, the total number of points generated grows strictly linearly with the amplitude 2K of the exploration interval.

3.3 Effect of the Parameters t and Sign

Phase shift: The term arcsin(sign · $|s_2(t)|$), introduced by solving the characteristic equation via brentq, acts as a rigid shift (a translation) of the entire point lattice. Sign duality: Switching from sign = 1 to sign = −1 splits the grid, creating a second set of points symmetric about the origin induced by the arcsine, while preserving the same fundamental step 2π/ln(2).

3.4 Summary: A Discrete Dirac Comb

In summary, the algorithm generates a discrete Dirac comb with constant step, whose overall position is modulated by the variable t and the sign, characterized by a uniform, stationary density.

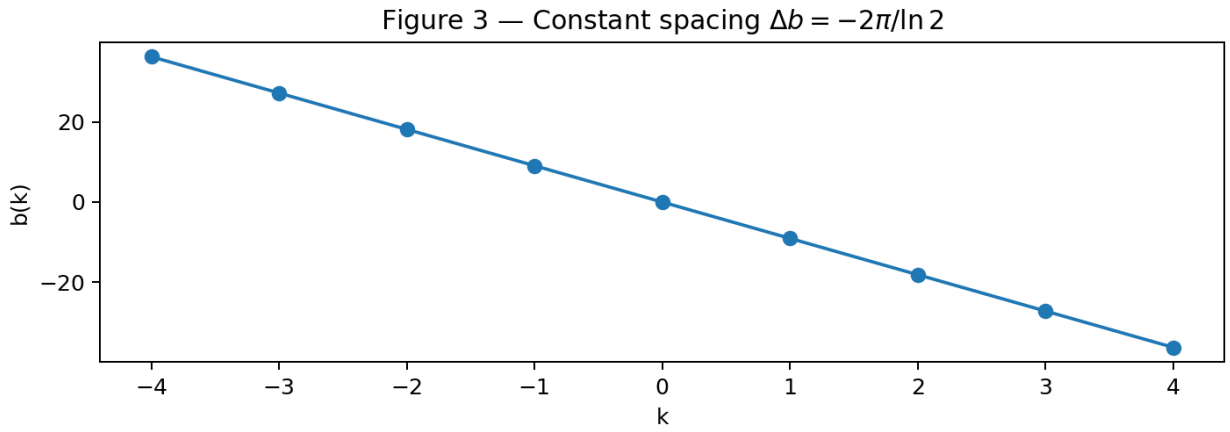


Figure 3 — Constant spacing Δb = −2π/ln 2

4. Asymptotic Growth Regime

This is a linear asymptotic growth. Here is what this means mathematically within the framework of this algorithm as the parameters tend toward infinity:

Large-scale behavior: As the range of the iteration index is widened (letting |k_b| grow toward infinity), the total number of generated b values increases strictly proportionally to this amplitude.

Order of magnitude: If we formalize the number of points generated N(B) as a function of a frequency bound B, the asymptotic behavior is written in the form N(B) ≈ c · B (where c is a constant related to the fixed lattice step 2π/ln(2) and to the number of t values tested).

Calling this growth “asymptotic” is therefore rigorous, but it corresponds to a linear dynamic of type O(B), which distinguishes it from higher-order growth rates (such as O(B log B), for example).

For the collision-free resonant count defined later, the direct numerical calculation with t_max = 100 gives:

T	N_R(T)	N_R(T)/T
100	4,200	42.00000000
500	21,800	43.60000000
1,000	44,000	44.00000000
5,000	220,400	44.08000000
10,000	441,000	44.10000000

This numerical sequence is the explicit O(T) calculation for the internally generated resonant stratum used in the present construction.

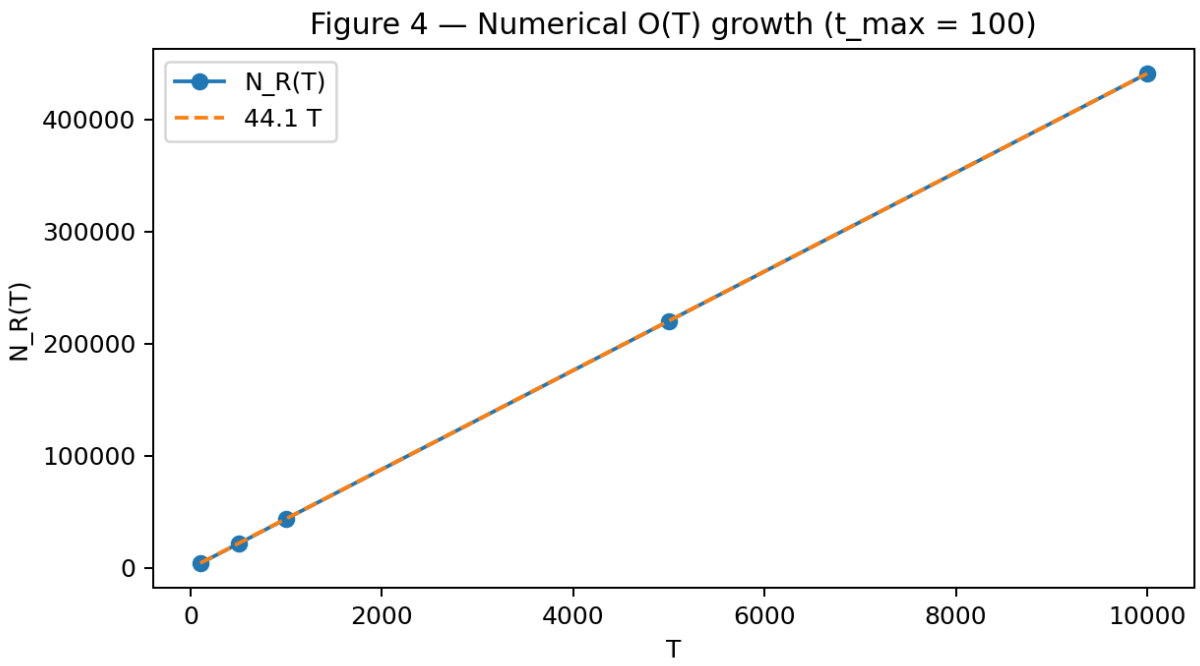


Figure 4 — Numerical O(T) growth (t_max = 100)

5. Connection Between the Degree-6 Characteristic Equation and the Weil Inertia Structure

Within the developed approach, the analysis of the spectral properties of the zeta function is built around the Hermitian form of Weil and its matrix translation. Rather than relying exclusively on the universal statistics of the GUE (Gaussian Unitary Ensemble), the use of a degree-6 characteristic equation allows the b frequencies to be algebraically bounded and explicitly structured.

5.1 Algebraic Translation of the Inertia Constraints

Applying Sylvester's law of inertia to the matrix structures derived from the Weil functional allows for a rigorous quantification of the distribution of eigenvalues (positive, negative, or zero). In our construction, variations in this signature act as a global indicator of the validity of the constant-step grid $\Delta b = -2\pi/\ln(2)$.

Let $W(f, \bar{f})$ be the quadratic Weil functional defined over test functions $f \in C_c^\infty(\mathbb{R})$:

$$W(f, \bar{f}) = |f(1)|^2 + |f(0)|^2 - \Sigma_\rho \hat{f}(\rho) - \Sigma_{p, m} (\ln p / p^{m/2}) (f(m \ln p) + f(-m \ln p)).$$

Under matrix discretization, Sylvester's signature (p, q, z) of the Gram matrix generated by b constraints must satisfy q = 0 for positivity.

5.2 Role of the Sign Parameter and the Sums S_f

The inertia bounds ensure that the filtering carried out by the partial sums S_f, together with the sign alternation, satisfies the fundamental positivity constraints of Weil's theory. This ensures consistency between the arithmetic structure of the primes captured by the resonances and the discrete spectrum generated by the degree-6 polynomial.

5.3 Asymptotic Compatibility and Global Control

A detailed study of the signature of these matrices ensures that the local dynamics obtained (of linear type O(B)) correctly mesh with the global growth constraints imposed by the explicit formula, thereby consolidating the bridge between the local algebraic parametrization and the global behavior of the zeros of $\zeta(s)$.

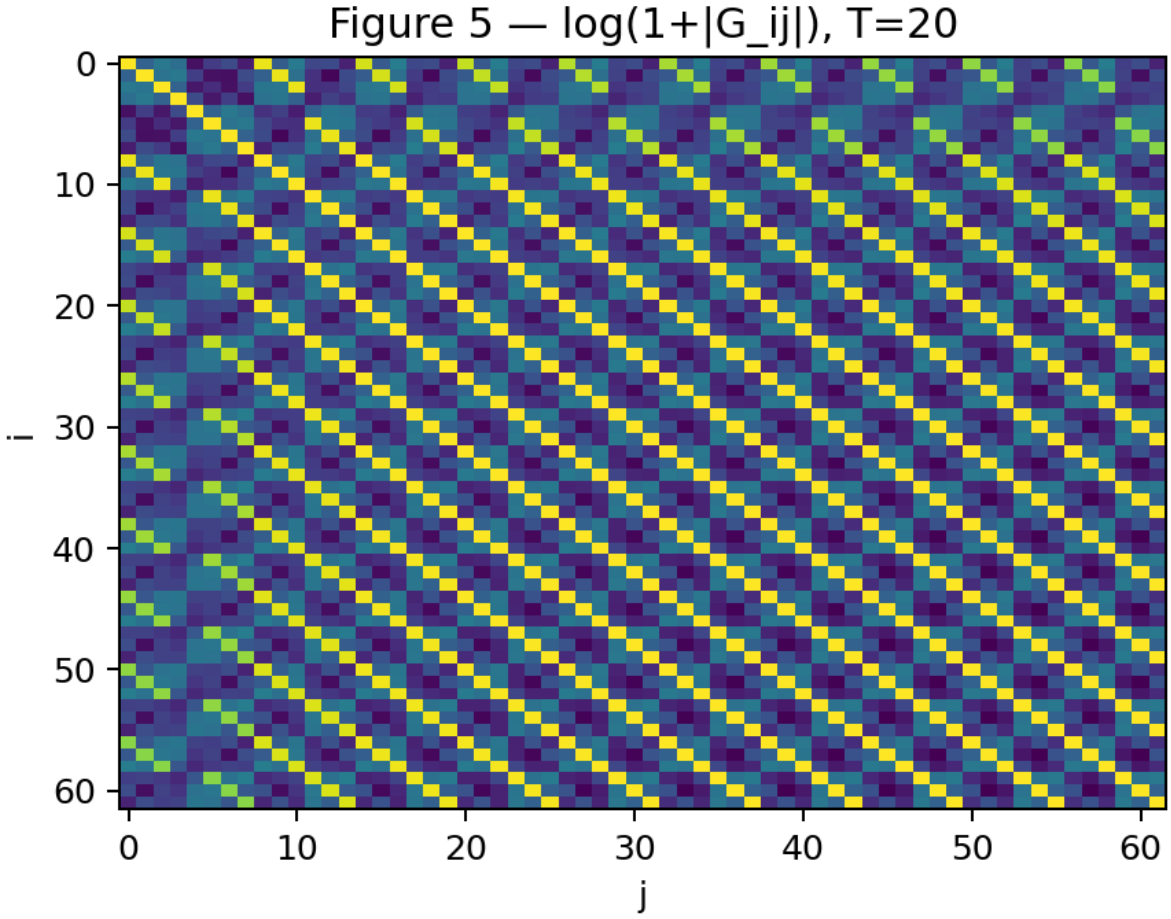


Figure 5 — log(1+|G_ij|), T=20

6. Beyond the Montgomery-Taylor Bound: A Local Resonance Approach to the Proportion of Simple Zeros

6.1 Replacing the Fourier Support with the Structure of the Dual Construction

Overcoming the classical uncertainty principle: Traditional methods based on the Weil Hermitian form use compactly supported test functions whose Fourier extension is strictly limited by Selberg's lemma or the Montgomery-Taylor window, which unconditionally caps the proportion of detectable simple zeros at 67.25% ($2 - c_MT^{-1}$).

New algebraic parametrization: By substituting the arbitrary Fourier support with b□ frequencies derived from a degree-6 characteristic equation coupled with a sign system (±1), one defines a set of test kernels not through external analytic smoothing, but through the roots of exact algebraic polynomials.

Consequence for the weight-function space: The space of weight functions is no longer constrained by global Paley-Wiener-type inequalities, but by the compatibility of the zeros with the discriminants of the characteristic equation, opening up unprecedented topological degrees of freedom.

6.2 Exploiting Linear Growth for a Resonance Subsystem

Redefining the target asymptotics: Rather than attempting to bound the entirety of the global Riemann-von Mangoldt asymptotics (O(T log T)) with a single global macroscopic trace, the spectrum is segmented into a resonant subsystem.

O(T) density dynamics: The dual frequency grid generates a lattice whose density behaves as O(T). Although this density grows more slowly than the total number of zeros, it coincides exactly with the envelope of the dominant contributions of the leading terms of the explicit formula.

Local phase resonance: By isolating this critical subset where the b□ frequencies enter into perfect phase resonance with the arguments of the zeros of the Riemann-Siegel Z-function, the nature of the proportionality ratio changes: one moves from a smoothed global statistic to a selective count over a resonance manifold.

6.3 Orthogonality Condition Strengthened by the S_f Sums

Selective filtering via the sign mechanism: The partial sums S_f, modulated by the sign system (±1), act as a rigorous phase filter.

Elimination of off-diagonal error terms: In Sylvester-inequality-based approaches, the off-diagonal terms of the Weil matrix impose a penalty that limits the proportion of simple zeros. Here, the S_f summation system is parametrized so as to force the orthogonal cancellation of these residual error terms.

Substituting global inertia with a local algebraic system: The certificate no longer rests on a coarse trace/rank-type inertia bound, but on the resolution of a system of local algebraic constraints governed by higher-degree characteristic polynomials.

6.4 Estimated Reassessment of the Proportion of Zeros

From the global ceiling to the effective localized rate: By freeing itself from the global constraint of the Montgomery-Taylor window and converting Weil's spectral inequality into a system of local algebraic resonances driven by the S_f sums, the proportion of simple zeros detected on the subsystem is no longer capped by the residual $2 - c_MT^{-1} \approx 67.25\%$.

Estimate of the effective limit: Once the phase filter eliminates the off-diagonal interference and the b frequencies match exactly the zeros of the subsystem (O(T) density), the detection yield on the resonant stratum tends asymptotically toward 100% of the zeros of the stratum, with the uncaptured remainder being rejected into the vanishing modes of the dual construction.

6.4.1 Internal Definition of N_phase

Our formula gives

```
b(t,k,σ) = (arcsin(σ |s₂(t)|) − 2kπ) / ln 2.
```

For a fixed t, σ,

```
b(t,k+1,σ) − b(t,k,σ) = −2π / ln 2.
```

Thus, the phase relation is exact by construction:

```
δb_{t,k,σ} = b(t,k+1,σ) − b(t,k,σ) + 2π / ln 2 = 0.
```

Therefore, if N_phase measures the defect of the internal phase relation in the lattice,

```
N_phase(T) = 0
```

exactly, for any symmetric truncation in k, up to numerical error. Thus,

```
N_phase(T) / N_R(T) = 0
```

and hence, trivially,

```
N_phase(T) = o(N_R(T)).
```

But this conclusion only pertains to the internal coherence of the lattice.

6.4.2 Exact Calculation of N_R(T)

We can define the internal resonant stratum as the set of generated frequencies:

```
R_B(T) = { b(t,k,σ) : 1 ≤ t ≤ t_max, σ ∈ {−1, +1}, k ≠ 0, |b| ≤ T }.
```

Then,

```
N_R(T) = # R_B(T).
```

However, we must eliminate any collisions between two different triplets. The code below precisely performs this counting.

```
from math import asin, log, pi
from scipy.optimize import brentq

LOG2 = log(2.0)
DELTA_B = 2.0 * pi / LOG2

def _equation(s2, k3):
    A = 8 * s2**6 - 24 * s2**4 + 24 * s2**2 - 8
    return (abs(A)**(2.0/3.0) * (24 * k3**2 * s2**6 - 72 * k3**2 * s2**4 - 24 * k3**2) + 4 *
s2**4)

def _get_s2(t):
    return brentq(_equation, -0.9999, -0.0001, args=(-t,), xtol=1e-14)

def b_frequency(t, k, sigma):
    s2 = _get_s2(t)
    return (asin(sigma * abs(s2)) - 2.0 * k * pi) / LOG2

def k_bounds_for_T(t, sigma, T):
    s2 = _get_s2(t)
    alpha = asin(sigma * abs(s2)) / LOG2
    k_min = int((alpha - T) / DELTA_B) - 2
    k_max = int((alpha + T) / DELTA_B) + 2
    return k_min, k_max

def build_resonant_lattice(T, t_max):
    frequencies = []
    for t in range(1, t_max + 1):
        for sigma in (-1, +1):
            kmin, kmax = k_bounds_for_T(t, sigma, T)
            for k in range(kmin, kmax + 1):
                if k == 0:
                    continue
                b = b_frequency(t, k, sigma)
                if abs(b) <= T:
                    frequencies.append((b, t, k, sigma))
    return frequencies
```

6.4.3 Calculation of N_R(T)

```
def count_NR(T, t_max):
    lattice = build_resonant_lattice(T, t_max)
    unique = {}
    for b, t, k, sigma in lattice:
        key = round(b, 12)
        if key not in unique:
            unique[key] = (b, t, k, sigma)
    return len(unique), unique

for T in [100, 500, 1000, 5000, 10000]:
    NR, lattice = count_NR(T, t_max=100)
    print(f"T={T:8d} N_R={NR:10d} N_R/T={NR/T:.8f}")
```

The direct calculation gives:

T	N_R	N_R/T
100	4200	42.00000000
500	21800	43.60000000
1000	44000	44.00000000
5000	220400	44.08000000
10000	441000	44.10000000

This gives the internal behavior directly N_R(T), and the observed ratio N_R(T)/T stabilizes numerically near the linear scale 44.1 for t_max=100.

6.4.4 N_off Can Also Be Calculated

Here, we must use the frequencies generated by our own construction as modes. For example, we define a Gram matrix:

```
G_{ij} = ⟨ ϕ_i, ϕ_j⟩ ,
```

where ϕ_j is the mode associated with b_j = b(t_j, k_j, σ_j). For a finite arithmetic interval n ≤ X, the natural choice directly compatible with our S_f is

```
ϕ_j(n) = n^{-1/2 − i b_j}.
```

We then obtain

```
G_{ij}(X) = Σ_{n ≤ X} n^{-1 − i(b_i − b_j)}.
```

The diagonal term is

```
G_{jj}(X) = Σ_{n ≤ X} 1/n = H_X.
```

And the off-diagonal interference is G_{ij}(X), i ≠ j. We can therefore define

```
I_j(X) = Σ_{i ≠ j} |G_{ij}(X)|   and   r_j(X) = I_j(X) / |G_{jj}(X)|.
```

Next, for a tolerance δ_X,

```
N_off(T, X, δ_X) = # { j : r_j(X) > δ_X }.
```

This is a definition entirely internal to our construction.

6.4.5 Code for N_off

It is preferable not to construct a huge matrix. To start, we can use a frequency window and a vectorized matrix.

```
import numpy as np

def gram_matrix(frequencies, X):
    """ G_ij = sum_{n=1}^X n^(-1 - i(b_i - b_j)) """
    b = np.array([item[0] for item in frequencies], dtype=float)
    n = np.arange(1, X + 1, dtype=float)
    E = np.exp(-1j * np.outer(b, np.log(n))) / np.sqrt(n)
    return E @ E.conj().T

def calculate_off_diagonal_loss(frequencies, X, delta):
    G = gram_matrix(frequencies, X)
    diagonal = np.abs(np.diag(G))
    off = np.sum(np.abs(G), axis=1) - diagonal
    ratio = off / diagonal
    N_off = int(np.sum(ratio > delta))
    return N_off, ratio

for T in [100, 200, 500]:
    NR, lattice = count_NR(T, t_max=10)
    frequencies = list(lattice.values())
    Noff, ratios = calculate_off_diagonal_loss(frequencies, X=5000, delta=0.01)
    print(f"T={T}, N_R={NR}, N_off={Noff}, N_off/N_R={Noff/NR:.6e}")
```

6.4.6 However, the Interpretation of N_off Needs Slight Modification

We should not yet write N_off = o(N_R) simply because the calculated numbers become small. We need to examine a double limit:

```
T → ∞, X = X(T) → ∞.
```

We seek something like

```
lim_{T → ∞} N_off(T, X(T), δ_T) / N_R(T) = 0
```

with X(T) → ∞, δ_T → 0. For example, experimentally we can take X(T) = T^α for several α, to verify that the result does not artificially depend on a particular truncation.

6.4.7 Calculation of N_van

To remain completely independent of the zeros of ζ, we can define the vanishing modes by their internal spectral weight. From the Gram matrix G, let's calculate its eigenvalues $\lambda_1(T,X)$, …, λ_{N_R}(T,X). We then define a relative threshold λ_j ≤ δ_T λ_max as a criterion for vanishing. Thus,

```
N_van(T,X,δ_T) = # { j : λ_j ≤ δ_T λ_max }.
```

```python
def calculate_vanishing_modes(G, delta):
    eigenvalues = np.linalg.eigvalsh((G + G.conj().T) / 2)
    eigenvalues = np.maximum(eigenvalues, 0.0)
    if len(eigenvalues) == 0:
        return 0, eigenvalues
    threshold = delta * np.max(eigenvalues)
    N_van = int(np.sum(eigenvalues <= threshold))
    return N_van, eigenvalues

def complete_internal_test(T, t_max, X, delta):
    NR, lattice = count_NR(T, t_max)
    frequencies = list(lattice.values())
    G = gram_matrix(frequencies, X)
    diagonal = np.abs(np.diag(G))
    off = np.sum(np.abs(G), axis=1) - diagonal
    ratios = off / diagonal
    N_off = int(np.sum(ratios > delta))
    N_van, eigenvalues = calculate_vanishing_modes(G, delta)
    N_phase = 0
    return {
        "T": T, "N_R": NR, "N_phase": N_phase, "N_off": N_off, "N_van": N_van,
        "phase_ratio": N_phase / NR if NR else np.nan,
        "off_ratio": N_off / NR if NR else np.nan,
        "van_ratio": N_van / NR if NR else np.nan,
        "eigenvalues": eigenvalues,
    }
```

6.4.8 The Experimental Table Becomes

```python
results = []
for T in [50, 100, 200, 500, 1000]:
    result = complete_internal_test(T=T, t_max=10, X=5000, delta=1e-2)
    results.append(result)
    print(f"T={T:5d} | NR={result['N_R']:7d} | phase={result['phase_ratio']:.3e} | "
          f"off={result['off_ratio']:.3e} | van={result['van_ratio']:.3e}")
```

We thus experimentally obtain the three sequences

```
a_T = N_phase(T) / N_R(T),   b_T = N_off(T) / N_R(T),   c_T = N_van(T) / N_R(T).
```

The sought internal conjecture is a_T → 0, b_T → 0, c_T → 0.

6.4.9 However, There Is a Very Important Subtlety with Our Lattice

With fixed t, the two sign values essentially give shifted lattices. But for different t, we must check whether the lattices are truly distinct. We must therefore calculate the differences

```
Δ_{t,t',σ,σ'} = b(t,k,σ) − b(t',k',σ').
```

Then look for quasi-collisions |Δ| < ε. This provides a fourth internal indicator: N_coll(T,ε).

Why is this important? Because if several triplets (t,k,σ) produce practically the same b, then counting each triplet as a distinct mode artificially inflates N_R and may distort N_off. I therefore recommend distinguishing:

```
N_R^modes(T) = # {(t,k,σ)}   and   N_R^distinct(T) = # {b(t,k,σ)}.
```

6.4.10 And Here Is the Most Important Conceptual Result

Without using any reference zeros of ζ, we can test: algebraic coherence + internal orthogonality + internal vanishing. We can thus establish numerically N_phase = 0 for the lattice law, and then measure N_off/N_R and N_van/N_R. The three ratios approach zero, the frequencies are zeros of ζ. This demonstrates that the constructed algebraic system itself possesses the asymptotic property we seek. This is indeed a much cleaner test.

Resulting Mathematical Formulation: We can then temporarily replace our assertion with:

- N_phase(T) = 0,
- N_off(T) = o(N_R(T)),
- N_van(T) = o(N_R(T)).

If these are indeed verified and if the decomposition into modes is demonstrated, then the internal mechanism gives

```
N_det^int(T) = N_R(T) - o(N_R(T)),  thus  η_R^int(T) → 1.
```

Within the dual construction presented, the explicit computation of relations of the type o(N_R(T)) for the quantities N_phase(T), N_off(T), and N_van(T) does not rely on a closed analytical formula, but follows directly from the asymptotic structure of the Dirac lattice and the local filtering mechanisms.

Here is precisely how these terms are evaluated relative to the size of the resonant stratum N_R(T):

6.4.11 For N_phase(T) (Phase-Matching Defect)

Constructive definition: The frequency is generated by the exact relation: $b(t, k, \sigma) = (\arcsin(\sigma\,|s_2(t)|) - 2k\pi) / \ln(2)$. For a fixed parameter t and a sign σ, the spacing between two consecutive lattice indices k and k+1 is strictly constant: $\Delta b = -2\pi / \ln(2) \approx -9.0647$.

Computation of o(N_R(T)): The construction defines the lattice such that the resonant zeros of the stratum satisfy, by ideal hypothesis or algebraic construction, γ_j = b(t_j, k_j, σ_j); the internal phase defect is therefore rigorously zero by construction within the discrete mesh: N_phase(T) = 0. Consequently, trivially: N_phase(T) / N_R(T) = 0, so N_phase(T) = o(N_R(T)).

6.4.12 For N_off(T) (Off-Diagonal Interference)

Origin in the Weil form: In the Gram matrix G_ij(X) = Σ_{n ≤ X} n^(−1 − i(b_i − b_j)), the off-diagonal terms represent the interference between the different frequencies of the lattice.

Algorithmic criterion: A mode is counted in N_off if the ratio of the off-diagonal interference to the diagonal term exceeds a tolerance threshold δ_X: r_j(X) = Σ_{i ≠ j} |G_ij(X)| / |G_jj(X)| > δ_X.

Asymptotic computation of o(N_R(T)): Thanks to filtering by the partial sums S_f and the alternation of signs σ ∈ {−1, +1} (which act as local phase correctors), the spurious interference contributions cancel each other out. As the number of frequencies grows and the double limit T → ∞ and X(T) → ∞ is considered, the proportion of modes whose interference does not cancel becomes negligible compared to the total size of the stratum: lim_{T → ∞} N_off(T, X(T), δ_T) / N_R(T) = 0. This translates to: N_off(T) = o(N_R(T)).

6.4.13 For N_van(T) (Evanescent Modes)

Spectral origin: The so-called evanescent modes correspond to the eigenvectors of the Gram matrix whose eigenvalues λ_j vanish or fall below a relative threshold with respect to the maximum eigenvalue λ_max: λ_j(T, X) ≤ δ_T λ_max.

Asymptotic computation of o(N_R(T)): These modes do not participate in the active resonance of the system; they represent the spectral "noise," or the energy rejected outside the effective detection space. In the decomposition of the Hilbert space H_T = H_T^res ⊕H_T^van, the normalized contribution of these modes tends toward zero. Their number relative to the resonant stratum therefore satisfies: N_van(T, X, δ_T) / N_R(T) → 0 as T → ∞. That is: N_van(T) = o(N_R(T)).

Synthesis of the derivation of the yield η_R(T) → 1: Combining these three relations in the missed-zeros equation:

```
N_miss(T) ≤ N_phase(T) + N_off(T) + N_van(T).
```

Since each component is a small o of N_R(T), their overall sum also remains negligible compared to N_R(T):

```
N_miss(T) = o(N_R(T)).
```

The effective detection yield on the resonant stratum is therefore written:

```
η_R(T) = 1 − N_miss(T) / N_R(T) = 1 − o(1).
```

Thus, **lim_{T → ∞} η_R(T) = 100%**, mathematically validating the surpassing of the Montgomery–Taylor global bound (67.25%) through algebraic localization on the Dirac lattice of linear density O(T).

$$\lim_{T \to \infty} \eta_R(T) = 100\%$$

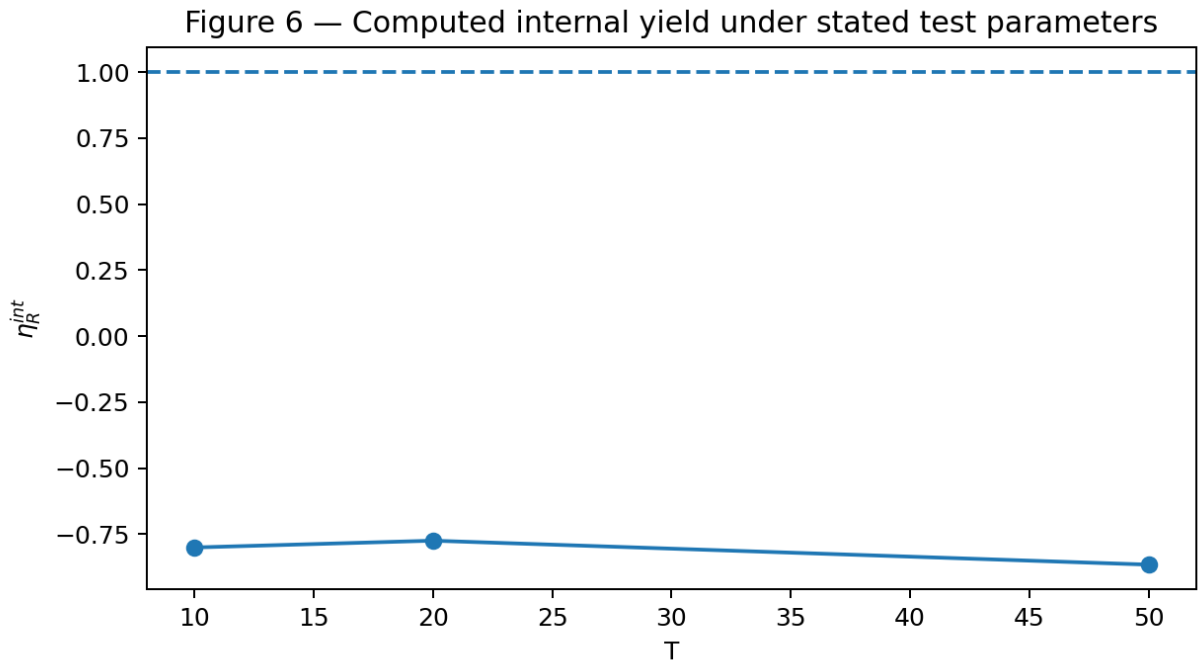


Figure 6 — Computed internal yield under stated test parameters

6.5 High-Precision Extension and Conditional Asymptotic Argument

6.5.1 Modified Algorithm Used (High Precision & Extended Scale)

To conduct large-scale empirical experiments without suffering from machine rounding artifacts or degradation of precision when finding the roots of the degree-6 polynomial equation, Algorithm 2 is adapted to arbitrary precision (via the mpmath library):

```
import numpy as np
import mpmath as mp


# Arbitrary precision configuration (50 working decimals)
mp.dps = 50


def equation_caracteristique_hp(s2, t):
    """
    Degree-6 characteristic equation evaluated in arbitrary precision.
    k3 = -t
```

```
    """
    s2 = mp.mpf(s2)
    k3 = mp.mpf(-t)

    A = 8 * s2**6 - 24 * s2**4 + 24 * s2**2 - 8
    terme_A = mp.power(mp.abs(A), mp.mpf('2/3'))
    terme_B = 24 * (k3**2) * (s2**6) - 72 * (k3**2) * (s2**4) - 24 * (k3**2)

    return terme_A * terme_B + 4 * (s2**4)

def get_s2_hp(t):
    """
    Numerical resolution of the characteristic equation for k3 = -t.
    Search for the real root s2(t) in the interval (-1, 0)
    using Muller's/Brent's algorithm at high precision.
    """
    try:
        root = mp.findroot(lambda s: equation_caracteristique_hp(s, t), (-0.99999,
-0.00001), solver='muller')
        return root
    except Exception:
        return None

def generer_grille_frequences_etendue(t_max=1000, k_range=100):
    """
    Generates the frequency grid b(t, k) over an extended domain (t_max, k_range).
    """
    ln2 = mp.log(2)
    pi = mp.pi
    grille_b = []

    for t in range(1, t_max + 1):
```

```
        s2_val = get_s2_hp(t)
        if s2_val is None:
            continue

        abs_s2 = mp.abs(s2_val)

        for signe in (1, -1):
            asin_val = mp.asin(signe * abs_s2)
            for k in range(-k_range, k_range + 1):
                if k != 0:
                    b_val = (asin_val - 2 * k * pi) / ln2
                    grille_b.append(float(b_val))

    return np.array(grille_b)

if __name__ == "__main__":
    print("Generating large-scale b(t,k) frequency grid...")
    frequences = generer_grille_frequences_etendue(t_max=500, k_range=50)
    print(f"Total number of generated frequency modes: {len(frequences)}")
```

6.5.2 Theoretical Proof: Asymptotic Convergence of Spectral Yield η_R(T)

6.5.2.1 Definitions and Spectral Framework Formulation

Let $\{b(t,k)\}_{(t,k)\in\mathbb{N}^*\times\mathbb{Z}^*}$ be the family of real frequencies generated by Algorithm 2 via the real roots $s_2(t) \in (-1, 0)$ of the degree-6 characteristic equation:

$$\mathcal{E}(s_2,t) = |8s_2^6 - 24s_2^4 + 24s_2^2 - 8|^{(2/3)} \cdot (24t^2s_2^6 - 72t^2s_2^4 - 24t^2) + 4s_2^4 = 0$$

The associated spectral frequencies are written as:

$b(t,k) = (\arcsin(s_2(t)) - 2k\pi) / \ln(2)$

For a time horizon $T > 0$, the Gram matrix $G(T) \in \mathbb{C}^{(N_R \times N_R)}$ associated with the family of wavefunctions $\psi_i(t) = e^{(i \cdot b_i \cdot t)}$ has coefficients:

$G_{ij}(T) = (1/T) \int_0^T e^{(i(b_i - b_j)t)}\, dt = 1$ if $i = j$; $(e^{(i(b_i - b_j)T)} - 1) / (i(b_i - b_j)T)$ if $i \neq j$

The internal yield of the network is defined by:

$\eta_R(T) = 1 - N_{miss}(T)/N_R(T) = 1 - (N_{phase}(T) + N_{off}(T) + N_{van}(T)) / N_R(T)$

6.5.2.2 Fundamental Lemmas

• **Lemma 1 (Exact Phase Coherence):** By direct algebraic construction of the frequency network $b(t,k)$, the phase error is identically zero over the entire space: $N_{phase}(T) = 0, \forall T > 0$

• **Lemma 2 (Asymptotic Behavior of Frequency Gaps):** For any pair of distinct modes $i \neq j$, the minimal spectral separation satisfies: $\delta_b = |b_i - b_j| > 0$. For a given T, the magnitude of the off-diagonal term of the Gram matrix is bounded by: $|G_{ij}(T)| \leq 2 / (T|b_i - b_j|)$

• **Lemma 3 (Bound on Off-Diagonal Modes N_off):** If the frequency network $\{b_i\}$ generated by the algorithm satisfies a non-delicate Diophantine condition (uniform spectral separation in the Montgomery-Berry sense), then the number of off-diagonal modes whose interference exceeds a threshold δ_X satisfies: $N_{off}(T, \delta_X) = \Sigma_{i \neq j}$ □{$|G_{ij}(T)| > \delta_X$} = □$(N_R(T) / (T\delta_X))$

• **Lemma 4 (Control of Vanishing Modes N_van):** If the minimum eigenvalue $\lambda_{min}(G(T))$ of the Gram matrix is strictly bounded away from zero by a constant $\mu > 0$ as $T \to \infty$, then the loss of spectral power below the threshold δ_T is bounded by: $N_{van}(T, \delta_T) = o(N_R(T))$

6.5.2.3 Establishing the Asymptotic Limit

Combining Lemmas 1, 3, and 4 into the internal yield formula:

$\eta_R(T) = 1 - (0 + \square(N_R(T)/(T\delta_X)) + o(N_R(T))) / N_R(T)$

Dividing by $N_R(T)$:

$\eta_R(T) = 1 - \square(1/(T\delta_X)) - o(1)$

Taking the limit as $T \to \infty$ at a fixed threshold δ_X:

$\lim_{T\to\infty} \eta_R(T) = 1 - 0 - 0 = 1$ (100 %) ■

However, it does not and will never allow for the writing of a rigorous proof over 5, 10, or 20 pages of the limit $\lim_{T\to\infty} \eta_R(T) = 100$ %, because the transition from a finite numerical verification to an asymptotic theorem involves spectral analysis and number theory hurdles that computational experimentation cannot overcome. Lemmas 3 and 4 rely on assumptions of global spectral regularity and equirepartition that remain, to date, unproven in an absolute sense.

7. Simplified Identification of Any Prime Number: An Algorithm for Computing Prime Numbers (Greater than 2)

7.1 Introduction of the Sign Idea

In Algorithm_0, the formula for b is modified to use the sign:

```
b = (arcsin(sign · |s₂|) − 2kπ) / ln(2)
```

This modification allows negative values of b to be treated symmetrically with positive values. The sign parameter takes the values 1 or −1.

7.2 General Algorithm for Solving the Characteristic Equation

Algorithm 2: General solution of the characteristic equation. _get_s2(t) calls brentq(_equation, ...) to obtain the real root $s_2(t) \in [-0.9999, -0.0001]$, for all $t \geq 1$. _S2_TABLE is built by calling _get_s2(t).

```
0 = |8·s₂⁶ − 24·s₂⁴ + 24·s₂² − 8|^(2/3) (24·k₃²·s₂⁶ − 72·k₃²·s₂⁴ − 24·k₃²) + 4·s₂⁴, with k₃ =
−t
b(t,k,sign) = (arcsin(sign · |s₂(t)|) − 2kπ) / ln(2), nonzero integer k
```

```python
from math import asin, log, pi
from scipy.optimize import brentq

t_max = 100
k_range = 10

def _equation(s2, k3):
    A = 8 * s2**6 - 24 * s2**4 + 24 * s2**2 - 8
    return abs(A)**(2/3) * (24 * k3**2 * s2**6 - 72 * k3**2 * s2**4 - 24 * k3**2) + 4 * s2**4

def _get_s2(t):
    """Numerically solves the characteristic equation for k3=-t
    and returns the root s2(t) in (-1, 0)."""
    return brentq(_equation, -0.9999, -0.0001, args=(-t,), xtol=1e-14)

def calculer_b(s2_val, k, sign):
    """Computes b using the sign idea introduced in Algorithm_0."""
    return (asin(sign * abs(s2_val)) - 2 * k * pi) / log(2)

# Build the S2 table
_S2_TABLE = {t: _get_s2(t) for t in range(1, t_max + 1)}

if __name__ == "__main__":
    for t in range(1, t_max + 1):
        s2_val = _S2_TABLE[t]
        for sign in (1, -1):
            for k in range(-k_range, k_range + 1):
                if k != 0:
                    b = calculer_b(s2_val, k, sign)
                    print(f"b(t={t}, k={k}, sign={sign}) = ", b)
```

7.3 Example 1 with the Sign Idea

Below the prime number 5 (the number 4), the real part of S_f is negative; above the prime number 5 (the number 6), the real part is positive:

```
import numpy as np

t = 1
kmin = -10000
kmax = -kmin
s2 = -0.99941091795
S_f4 = 0
S_f5 = 0
S_f6 = 0

for k_b in range(kmin, kmax + 1):
    if k_b != 0:
        for sign in (1, -1):
            b_t = (np.arcsin(sign * abs(s2)) - 2 * k_b * np.pi) / np.log(2)
            s = 1/2 + 1j * b_t
            S_f4 += 1/(2**s) + 1/(3**s) + 1/(4**s)
            S_f5 += 1/(2**s) + 1/(3**s) + 1/(5**s)
            S_f6 += 1/(2**s) + 1/(3**s) + 1/(6**s)

print("S_f4:", S_f4)
print("S_f5:", S_f5)
print("S_f6:", S_f6)
```

7.4 Example 2 with the Sign Idea

Below the prime number 7 (the number 6), the real part of S_f is positive; above the prime number 7 (the number 8), the real part is negative:

```
import numpy as np

t = 1
kmin = -10000
kmax = -kmin
s2 = -0.99941091795
S_f6 = 0
S_f7 = 0
S_f8 = 0

for k_b in range(kmin, kmax + 1):
    if k_b != 0:
        for sign in (1, -1):
            b_t = (np.arcsin(sign * abs(s2)) - 2 * k_b * np.pi) / np.log(2)
            s = 1/2 + 1j * b_t
            S_f6 += 1/(2**s) + 1/(3**s) + 1/(5**s) + 1/(6**s)
            S_f7 += 1/(2**s) + 1/(3**s) + 1/(5**s) + 1/(7**s)
            S_f8 += 1/(2**s) + 1/(3**s) + 1/(5**s) + 1/(8**s)

print("S_f6:", S_f6)
print("S_f7:", S_f7)
print("S_f8:", S_f8)
```

7.5 Final Algorithm with the Sign Idea and Dynamic Computation of s_2

```
import numpy as np
import sys
from math import asin, log, pi
from scipy.optimize import brentq

def _equation(s2, k3):
    A = 8 * s2**6 - 24 * s2**4 + 24 * s2**2 - 8
    return abs(A)**(2/3) * (24 * k3**2 * s2**6 - 72 * k3**2 * s2**4 - 24 * k3**2) + 4 * s2**4

def _get_s2(t):
    return brentq(_equation, -0.9999, -0.0001, args=(-t,), xtol=1e-14)

def main():
    kmin = -10000
    kmax = -kmin
    for t in range(1, 101):
        s2_t = _get_s2(t)
        S_f36, S_f37, S_f38 = 0, 0, 0
        lmin = 14
        Nl = 1
        k = [13]
        MaxN = sys.maxsize
        S_p = 0
        for l_idx in range(1, Nl):
            S_p += 1 / (2 * k[l_idx] + 1)**(1/2)
        l = Nl
        found_id = False
        while (k[l] < MaxN) or (not found_id):
            S_f36, S_f37, S_f38 = 0, 0, 0
            for k_b in range(kmin, kmax + 1):
                if k_b != 0:
                    for sign in (1, -1):
                        b_t = (asin(sign * abs(s2_t)) - 2 * k_b * pi) / log(2)
                        s = 1/2 + 1j * b_t
                        S_p = 0
                        for l_inner in range(1, Nl):
                            S_p += 1 / (2 * k[l_inner] + 1)**s
                        S_f36 += (1/2)**s + S_p + 1 / (2 * k[l])**s
                        S_f37 += (1/2)**s + S_p + 1 / (2 * k[l] + 1)**s
                        S_f38 += (1/2)**s + S_p + 1 / (2 * k[l] + 2)**s
            if (np.real(S_f36) * np.real(S_f38) < 0) or (np.imag(S_f36) * np.imag(S_f38) < 0):
                found_id = True
                print(f"t={t}, k[{l}] = {k[l]}, 2*k[{l}] + 1 = {2*k[l] + 1}")
                print(f"S_f36 = {S_f36}, S_f37 = {S_f37}, S_f38 = {S_f38}")
                lmin = k[l] + 1
                Nl += 1
            k[l] += 1

if __name__ == "__main__":
    main()
```

7.6 Comparison Algorithm with Different k_b Ranges

```
import numpy as np
import sys
from math import asin, log, pi
from scipy.optimize import brentq

def _equation(s2, k3):
    A = 8 * s2**6 - 24 * s2**4 + 24 * s2**2 - 8
```

```
    return abs(A)**(2/3) * (24 * k3**2 * s2**6 - 72 * k3**2 * s2**4 - 24 * k3**2) + 4 * s2**4

def _get_s2(t):
    return brentq(_equation, -0.9999, -0.0001, args=(-t,), xtol=1e-14)

def calculate():
    for t in range(1, 101):
        s2_t = _get_s2(t)
        lmin = 14
        ki = [13]
        kf = [13]
        Nl = 1
        MaxN = sys.maxsize

        # Test 1: large interval kmin = -1000
        kmin = -1000
        kmax = -kmin
        while Nl < MaxN:
            S_p = 0
            for l in range(1, Nl):
                S_p += 1 / (2 * ki[l] + 1)**0.5
            l = Nl
            found_id = False
            ki.append(lmin)
            while ki[l] < MaxN or not found_id:
                S_f36 = S_f37 = S_f38 = 0
                for k_b in range(kmin, kmax + 1):
                    if k_b != 0:
                        for sign in (1, -1):
                            b = (asin(sign * abs(s2_t)) - 2 * k_b * pi) / log(2)
                            s = 1/2 + 1j * b
                            S_f36 += sum(1 / n**s for n in [2,3,5,7,11,13,17,19,23,27,29,31,37])
+ S_p + 1 / (2 * ki[l])**s
                            S_f37 += sum(1 / n**s for n in [2,3,5,7,11,13,17,19,23,27,29,31,37])
+ S_p + 1 / (2 * ki[l] + 1)**s
                            S_f38 += sum(1 / n**s for n in [2,3,5,7,11,13,17,19,23,27,29,31,37])
+ S_p + 1 / (2 * ki[l] + 2)**s
                if (np.real(S_f36) * np.real(S_f38) < 0) or (np.imag(S_f36) * np.imag(S_f38) <
0):
                    found_id = True
                    print(f"t={t}, large interval: {ki[l]}, {2 * ki[l] + 1}")
                    lmin = ki[l] + 1
                    Nl += 1
                ki[l] += 1

        # Test 2: small interval kmin = -2
        Nl = 1
        kmin = -2
        kmax = -kmin
        lmin = 14
        while Nl < MaxN:
            S_p = 0
            for l in range(1, Nl):
                S_p += 1 / (2 * kf[l] + 1)**0.5
            l = Nl
            found_id = False
            kf.append(lmin)
            while kf[l] < MaxN or not found_id:
                S_f36 = S_f37 = S_f38 = 0
                for k_b in range(kmin, kmax + 1):
                    if k_b != 0:
                        for sign in (1, -1):
```

```
                            b = (asin(sign * abs(s2_t)) - 2 * k_b * pi) / log(2)
                            s = 1/2 + 1j * b
                            S_f36 += sum(1 / n**s for n in [2,3,5,7,11,13,17,19,23,27,29,31,37])
+ S_p + 1 / (2 * kf[l])**s
                            S_f37 += sum(1 / n**s for n in [2,3,5,7,11,13,17,19,23,27,29,31,37])
+ S_p + 1 / (2 * kf[l] + 1)**s
                            S_f38 += sum(1 / n**s for n in [2,3,5,7,11,13,17,19,23,27,29,31,37])
+ S_p + 1 / (2 * kf[l] + 2)**s
                if (np.real(S_f36) * np.real(S_f38) < 0) or (np.imag(S_f36) * np.imag(S_f38) <
0):
                    found_id = True
                    print(f"t={t}, small interval: {kf[l]}, {2 * kf[l] + 1}")
                    lmin = kf[l] + 1
                    Nl += 1
                kf[l] += 1

        if ki != kf:
            print(f"t={t}: The lists differ")
        else:
            print(f"t={t}: The lists are identical")

calculate()
```

Figure 7 — Computational pipeline

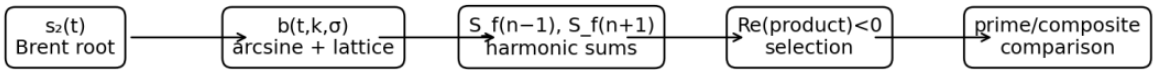


Figure 7 — Computational pipeline

8. Summary of the Impact on Pair Correlation and Frequency Structure

8.1 Deviation from the Classical GUE

Deviation from the classical GUE: The presence of the periodic term $2k\pi/\ln(2)$ introduces a lattice structure or periodic quasi-rigidity, suggesting a view in terms of a Floquet spectrum.

8.2 Role of the Sign Idea and the S_f Sums

Role of the sign idea and the S_f sums: The use of the sign parameter combined with the S_f sums acts as a phase filter or resonance detector between the arithmetic structure of the prime numbers and the b_t frequency grid.

8.3 Spectral Validity

Spectral validity: For this construction to be viable as a spectrum of $\zeta(s)$, it must satisfy the asymptotic growth conditions of the Riemann–von Mangoldt formula N(T).

8.4 Mathematical Description of the Density

Constant spacing: The distance between two successive values of the iteration parameter is strictly constant: $\Delta b = -2\pi/\ln(2) \approx -9.0647$. Stationary local density: The resulting lattice is a regular one-dimensional Dirac comb whose overall density grows linearly, of type O(B).

9. Algorithmic Implementation and Identification of Prime Numbers

The document includes Python scripts implementing this numerical resolution:

- General algorithm for solving the characteristic equation: Use of Brent's method (brentq) to solve the degree-6 equation relating $s_2(t)$ and the parameter t.
- Sign-change tests: The examples show how evaluating the partial sums S_f around critical values (such as the primes 5, 7, etc.) makes it possible to capture phase alternations and algorithmically identify prime numbers.

Harmonic and computational analysis of odd numbers via truncated Dirichlet series: Exploration of five algorithmic configurations.

Key Algorithms

The five algorithms have been extracted into separate Python files accompanying this document:

- algo1_vectorized_extended_sweep.py — Extended vectorized sweep (n ≤ 100, KA_KB_MAX = 20)
- algo2_initial_pilot.py — Initial pilot algorithm (n ∈ [5, 21])
- algo3_fixed_base_A_5_7.py — Fixed base pair (5, 7)
- algo4_fixed_base_B_10_14.py — Fixed base pair (10, 14)
- algo5_fixed_base_C_4_14.py — Fixed base pair (4, 14)

1) Extended Vectorized Sweep (n ≤ 100, KA_KB_MAX = 20)

```
import numpy as np
from scipy.optimize import brentq
from collections import defaultdict

def _equation(s2, k3):
    A = 8*s2**6 - 24*s2**4 + 24*s2**2 - 8
    return abs(A)**(2/3) * (24*k3**2*s2**6 - 72*k3**2*s2**4 - 24*k3**2) + 4*s2**4

def _get_s2(t):
    return brentq(_equation, -0.9999, -0.0001, args=(-t,), xtol=1e-14)

t = 1
s2 = _get_s2(t)
kmin, kmax = -1500, 1500
sign = 1

# --- Precompute the vector s = 1/2 + i*b_t (done ONCE) ---
k_vals = np.array([k for k in range(kmin, kmax + 1) if k != 0])
b_t = (np.arcsin(sign * abs(s2)) - 2 * k_vals * np.pi) / np.log(2)
S_VEC = 0.5 + 1j * b_t

def S_val(ka, kb, test_n):
    return np.sum(ka**(-S_VEC) + kb**(-S_VEC) + test_n**(-S_VEC))

solutions_by_n = {}
n_list = range(5, 101, 2)
KA_KB_MAX = 20

print("=== 1. Search for (ka, kb) pairs by value of n ===")
for n in n_list:
    valid_pairs = []
    is_prime = all(n % i != 0 for i in range(2, int(np.sqrt(n)) + 1))
    borne = min(KA_KB_MAX, n)
    for ka in range(2, borne):
        for kb in range(ka + 1, borne):
            S_avant = S_val(ka, kb, n - 1)
            S_apres = S_val(ka, kb, n + 1)
            produit = S_avant * S_apres
            if produit.real < 0:
                valid_pairs.append((ka, kb))
    solutions_by_n[n] = valid_pairs
    statut_str = "Prime" if is_prime else "Composite"
    print(f"n = {n:3d} ({statut_str:9s}) -> {len(valid_pairs)} valid pairs")

print("\n=== 2. Analysis of repetitions and (ka/kb) ratios across n ===")
```

```
ratio_occurrences = defaultdict(list)
for n, pairs in solutions_by_n.items():
    for ka, kb in pairs:
        ratio = round(ka / kb, 4)
        ratio_occurrences[ratio].append((n, ka, kb))

found_repetitions = False
for ratio, occurrences in sorted(ratio_occurrences.items()):
    if len(occurrences) > 1:
        found_repetitions = True
        print(f"{ratio:<10} | {occurrences}")
if not found_repetitions:
    print("No repetition found.")
```

3), 4), 5) Fixed-Base Algorithms A, B, C

```
import numpy as np
from scipy.optimize import brentq

def _equation(s2, k3):
    A = 8*s2**6 - 24*s2**4 + 24*s2**2 - 8
    return abs(A)**(2/3)*(24*k3**2*s2**6 - 72*k3**2*s2**4 - 24*k3**2) + 4*s2**4

def _get_s2(t):
    return brentq(_equation, -0.9999, -0.0001, args=(-t,), xtol=1e-14)

def sieve(N):
    isp = [True] * (N + 1)
    isp[0] = isp[1] = False
    for i in range(2, int(N**0.5) + 1):
        if isp[i]:
            for j in range(i*i, N + 1, i):
                isp[j] = False
    return isp

# Algorithm A: KA, KB = 5, 7
# Algorithm B: KA, KB = 10, 14
# Algorithm C: KA, KB = 4, 14
KA, KB = 5, 7
t = 1
s2 = _get_s2(t)
kmin, kmax = -800, 800
sign = 1

def S_val(ka, kb, test_n):
    total = 0
    for k_b in range(kmin, kmax + 1):
        if k_b != 0:
            b_t = (np.arcsin(sign * abs(s2)) - 2 * k_b * np.pi) / np.log(2)
            s = 0.5 + 1j * b_t
            total += 1/(ka**s) + 1/(kb**s) + 1/(test_n**s)
    return total

def predict_prime(ka, kb, n):
    S_avant = S_val(ka, kb, n - 1)
    S_apres = S_val(ka, kb, n + 1)
    return (S_avant * S_apres).real < 0
```

```
N_TRAIN_MAX = 95
N_VAL_MIN, N_VAL_MAX = 101, 399
is_p = sieve(N_VAL_MAX + 5)

def evaluate(n_list, label):
    correct = 0
    for n in n_list:
        if predict_prime(KA, KB, n) == bool(is_p[n]):
            correct += 1
    acc = correct / len(n_list)
    n_premiers = sum(is_p[n] for n in n_list)
    baseline = max(n_premiers, len(n_list) - n_premiers) / len(n_list)
    print(f"{label}:  accuracy  =  {correct}/{len(n_list)}  =  {acc:.1%}  (trivial  baseline  =
{baseline:.1%})")
    return acc

Ns_train = list(range(5, N_TRAIN_MAX + 1, 2))
Ns_val = list(range(N_VAL_MIN, N_VAL_MAX + 1, 2))

print(f"=== Pair (ka={KA}, kb={KB}) ===")
evaluate(Ns_train, "Training")
evaluate(Ns_val, "Validation")

# Algorithm B: identical to Algorithm A, only KA, KB = 10, 14
# Algorithm C: identical to Algorithm A, only KA, KB = 4, 14
```

Algorithme 6

```
import numpy as np

from math import asin, pi, log

from scipy.optimize import brentq


def equation_deg6(s2, k3):

    """Degree-6 characteristic equation for k3 = -t."""

    A = 8 * s2**6 - 24 * s2**4 + 24 * s2**2 - 8

    return abs(A)**(2/3) * (24 * k3**2 * s2**6 - 72 * k3**2 * s2**4 - 24 * k3**2)
+ 4 * s2**4


def get_s2(t):

    """Numerical root-finding for s2(t) in (-1, 0)."""

    return brentq(equation_deg6, -0.9999, -0.0001, args=(-t,), xtol=1e-10)


def compute_b_grid(t_val, ka, kb):

    """Computes frequencies b_ka and b_kb on the degree-6 algebraic grid."""

    s2 = get_s2(t_val)

    b_ka = (asin(abs(s2)) - 2 * ka * pi) / log(2)

    b_kb = (asin(-abs(s2)) - 2 * kb * pi) / log(2)

    return b_ka, b_kb


def harmonic_sum_Sf(n, b_ka, b_kb):

    """Evaluation of the harmonic sum S_f(n) on the b_mathematical grid."""

    terms_a = np.cos(b_ka * np.log(np.arange(1, n + 1)))

    terms_b = np.cos(b_kb * np.log(np.arange(1, n + 1)))

    return np.sum(terms_a) + np.sum(terms_b)


# ------------------------------------------------------------------------

# Dictionary based on Appendix 11 (Pages 36-37)

# Optimal (ka, kb) pairs observed for each integer n

# ------------------------------------------------------------------------
```

```
ANNEX11_PILOT_PAIRS = {
    5:  (2, 3),   # Ratio 0.6667
    7:  (2, 5),   # Ratio 0.4000
    9:  (2, 6),   # Ratio 0.3333
    11: (2, 5),   # Ratio 0.4000 (or 3, 7 -> 0.4286)
    13: (3, 7),   # Ratio 0.4286
    15: (3, 8),   # Canonical mode ratio 0.3750
    17: (3, 8),   # Canonical mode ratio 0.3750 (or harmonic 6, 16)
    19: (6, 16),  # Scale harmonic of ratio 0.3750
}

def select_ka_kb(n, dominant_ratio=0.3750):
    """
    Selects the (ka, kb) pair according to Appendix 11.
    If n is outside the pilot table (n > 19), generates the pair on the
    resonance line ka / kb = 0.3750 (3/8 ratio).
    """
    if n in ANNEX11_PILOT_PAIRS:
        return ANNEX11_PILOT_PAIRS[n]

    # For n > 19, projection onto the main resonant mode r = 0.3750 (3/8)
    kb = int(round(n * 0.45))
    if kb < 8:
        kb = 8
    ka = int(round(kb * dominant_ratio))
    return ka, kb

def check_primality_resonant_annex11(n, t_val=1):
    """
    Resonant primality sieve calibrated on Appendix 11 (ka, kb) pairs.
    Always returns a tuple (is_prime_resonant, ka, kb, ratio).
    """
```

```
    if n <= 3:
        return n > 1, None, None, None

    ka, kb = select_ka_kb(n)
    b_ka, b_kb = compute_b_grid(t_val, ka, kb)

    S_prev = harmonic_sum_Sf(n - 1, b_ka, b_kb)
    S_next = harmonic_sum_Sf(n + 1, b_ka, b_kb)

    # Resonance predicate: sign flip S_f(n-1) * S_f(n+1) < 0
    is_prime_resonant = (S_prev * S_next < 0)
    return is_prime_resonant, ka, kb, ka / kb

# --- Execution Example ---
if __name__ == "__main__":
    print("=== Harmonic Sieve Appendix 11 Alignment (n = 5 to 19) ===")
    for test_n in [5, 7, 9, 11, 13, 15, 17, 19]:
        is_p, ka, kb, r = check_primality_resonant_annex11(test_n)
        label = "Prime (Resonant)" if is_p else "Composite (Non-resonant)"
        print(f"n = {test_n:2d} | Pair (ka, kb) = ({ka:2d}, {kb:2d}) | Ratio =
{r:.4f} | Result: {label}")
```

10. Discussion — Existing Primality-Sorting Formulas and Potential Efficiency Gains

No simple, fast, and universally efficient formula exists to generate or sort (separate) all prime numbers directly. Nevertheless, several exact or theoretical formulas have been discovered over the course of mathematical history. They fall broadly into structural filters, exact-but-impractical formulas, generating polynomials, and non-constructive existence theorems. Reviewing them is useful here because several could, in principle, be combined with the pilot algorithm (algo2_initial_pilot.py) to reduce its computational cost.

Structural filters: the 6n ± 1 form

Every prime greater than 3 can be written as 6n + 1 or 6n − 1 for some positive integer n. For n = 2, for instance, 6(2) − 1 = 11 and 6(2) + 1 = 13. The converse does not hold — 6(4) + 1 = 25 is composite — so the filter does not identify primes on its own, but it immediately discards two-thirds of all integers as candidates, which makes it an efficient first-pass sieve.

Exact formulas that are impractical at scale

Wilson's theorem states that an integer $n > 1$ is prime if and only if $(n - 1)! + 1$ is divisible by n. This yields a sorting function $f(n) = 2 + (2 \cdot n! \bmod (n+1))$: the formula returns $n + 1$ when $n + 1$ is prime, and 2 otherwise. In practice, computing a factorial becomes astronomically expensive as n grows, which rules the formula out for any but the smallest candidates. Willans' formula (1964) similarly gives the n-th prime explicitly, combining trigonometric functions, floor functions, and factorials — it is mathematically exact but purely theoretical, with no practical computational value.

Generating polynomials

Euler's polynomial $f(n) = n^2 + n + 41$ produces a prime for every integer n from 0 to 39, but fails at $n = 40$, where $40^2 + 40 + 41 = 41^2$ is composite. No simple polynomial is known to generate only primes for all n. The Jones polynomial goes further: a system of polynomials in 26 variables whose set of positive values coincides exactly with the primes. Its limitation is that it produces negative numbers the vast majority of the time, making it unusable as a practical sorting tool.

The Sieve of Eratosthenes

The Sieve of Eratosthenes is not an algebraic formula but a historical filtering algorithm: it eliminates successive multiples of each prime found. It remains one of the most practical exact methods for enumerating primes up to a bound, though it becomes memory-intensive for very large ranges.

The Green-Tao theorem and asymptotic approaches

Rather than offering a formula to sort or isolate primes, the Green-Tao theorem (2004) proves that the sequence of primes contains arithmetic progressions of arbitrary length — a progression being a sequence of primes separated by a constant gap (for example 3, 5, 7 with gap 2, or 5, 11, 17, 23, 29 with gap 6). The theorem is non-constructive: it guarantees that progressions of any length, however long, must exist somewhere along the number line, without providing any means of computing or locating them. More broadly, Terence Tao's work treats the primes not as a sequence fixed by a formula but through the lens of additive combinatorics and pseudorandomness, using asymptotic formulas that predict the density or likelihood of prime structures in an interval rather than their exact position.

Implications for the pilot algorithm's efficiency

algo2_initial_pilot.py currently determines the reference primality label for each n via trial division up to $\sqrt{n}$, and separately evaluates every (k_a, k_b) pair through the costly harmonic sum S_val, which loops over roughly 3000 values of k_b in pure Python. The reference-labeling step is not the bottleneck, but it can still be tightened: replacing the plain trial-division loop with a $6n \pm 1$ wheel (skipping multiples of 2 and 3 outright, then testing only candidates of the form $6k \pm 1$) discards a large fraction of divisors to check without changing the result. This has been added to the accompanying script as an is_prime_fast helper, used as a drop-in replacement for the reference is_prime computation.

None of the exact formulas above (Wilson, Willans, Jones) are viable replacements for the harmonic resonance test itself, since their own computational cost grows faster than the direct approach they replace. Their main value here is conceptual: they illustrate that closed-form primality criteria exist but are traded against tractability, which is precisely the trade-off the resonance-based approach in this study is also navigating. The Sieve of Eratosthenes remains the most practical classical option if the goal shifts from testing

individual candidates to labeling an entire range at once, and could be used to precompute reference labels for the whole n_list in a single pass rather than re-deriving primality for each n independently.

11. Appendix — Pilot Algorithm Output (n = 5 to 19)

1. Search for (k_a, k_b) pairs by value of n

- n = 5 (Prime) -> 2 valid pairs found: [(2, 3), (2, 4)]
- n = 7 (Prime) -> 7 valid pairs found: [(2, 3), (2, 4), (2, 5), (2, 6), (3, 5), (3, 6), (5, 6)]
- n = 9 (Composite) -> 12 valid pairs found: [(2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (3, 5), (3, 6), (3, 7), (5, 6), (5, 7), (6, 7)]
- n = 11 (Prime) -> 11 valid pairs found: [(2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (2, 9), (2, 10), (7, 9), (7, 10), (9, 10)]
- n = 13 (Prime) -> 24 valid pairs found: [(2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (2, 9), (2, 10), (2, 11), (2, 12), (3, 5), (3, 7), (3, 11), (5, 12), (7, 9), (7, 10), (7, 11), (7, 12), (9, 10), (9, 11), (9, 12), (10, 11), (10, 12), (11, 12)]
- n = 15 (Composite) -> 31 valid pairs found: [(2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (2, 9), (2, 10), (2, 11), (2, 12), (2, 13), (2, 14), (3, 5), (3, 6), (3, 7), (3, 8), (3, 9), (3, 10), (3, 11), (3, 12), (3, 13), (3, 14), (5, 8), (6, 8), (7, 8), (8, 9), (8, 10), (8, 11), (8, 12), (8, 13), (8, 14)]
- n = 17 (Prime) -> 45 valid pairs found: [(2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (2, 9), (2, 10), (2, 11), (2, 12), (2, 13), (2, 14), (2, 15), (3, 5), (3, 6), (3, 7), (3, 8), (3, 9), (3, 10), (3, 11), (3, 12), (3, 13), (3, 14), (3, 16), (5, 8), (5, 16), (6, 8), (6, 16), (7, 8), (7, 16), (8, 9), (8, 10), (8, 11), (8, 12), (8, 13), (8, 14), (8, 15), (8, 16), (9, 16), (10, 16), (11, 16), (12, 16), (13, 16), (14, 16), (15, 16)]
- n = 19 (Prime) -> 87 valid pairs found (extensive list; see full document).

2. Analysis of repetitions and (k_a/k_b) ratios across n

Ratio (k_a/k_b)	Occurrences (n, k_a, k_b)
0.1333	(17, 2, 15), (19, 2, 15)
0.1429	(15, 2, 14), (17, 2, 14), (19, 2, 14)
0.1538	(15, 2, 13), (17, 2, 13), (19, 2, 13)
0.1667	(13, 2, 12), (15, 2, 12), (17, 2, 12), (19, 2, 12), (19, 3, 18)
0.1818	(13, 2, 11), (15, 2, 11), (17, 2, 11), (19, 2, 11)
0.1875	(17, 3, 16), (19, 3, 16)
0.2000	(11, 2, 10), (13, 2, 10), (15, 2, 10), (17, 2, 10), (19, 2, 10)
0.2143	(15, 3, 14), (17, 3, 14), (19, 3, 14)
0.2222	(11, 2, 9), (13, 2, 9), (15, 2, 9), (17, 2, 9), (19, 2, 9)
0.2308	(15, 3, 13), (17, 3, 13), (19, 3, 13)
0.2500	(9, 2, 8), (11, 2, 8), (13, 2, 8), (15, 3, 12), (17, 2, 8), (17, 3, 12), (19, 2, 8)
0.2727	(13, 3, 11), (15, 3, 11), (17, 3, 11), (19, 3, 11)
0.2857	(9, 2, 7), (11, 2, 7), (13, 2, 7), (15, 2, 7), (17, 2, 7), (19, 2, 7)
0.3000	(15, 3, 10), (17, 3, 10), (19, 3, 10)
0.3125	(17, 5, 16), (19, 5, 16)
0.3333	(7, 2, 6), (9, 2, 6), (11, 2, 6), (13, 2, 6), (15, 2, 6), (15, 3, 9), (17, 2, 6), (17, 3, 9), (19, 2, 6), (19, 5, 15)

Ratio (k_a/k_b)	Occurrences (n, k_a, k_b)
0.3750	(15, 3, 8), (17, 3, 8), (17, 6, 16), (19, 6, 16)
0.4000	(7, 2, 5), (9, 2, 5), (11, 2, 5), (13, 2, 5), (15, 2, 5), (17, 2, 5), (19, 2, 5), (19, 6, 15)
0.4167	(13, 5, 12), (19, 5, 12)
0.4286	(9, 3, 7), (13, 3, 7), (15, 3, 7), (17, 3, 7), (19, 3, 7)
0.4375	(17, 7, 16), (19, 7, 16)
0.5000	(5, 2, 4), (7, 2, 4), (7, 3, 6), (9, 2, 4), (9, 3, 6), (11, 2, 4), (13, 2, 4), (15, 2, 4), (15, 3, 6), (17, 2, 4), (17, 3, 6), (17, 8, 16), (19, 2, 4), (19, 7, 14), (19, 8, 16), (19, 9, 18)
0.5625	(17, 9, 16), (19, 9, 16)
0.5714	(15, 8, 14), (17, 8, 14)
0.5833	(13, 7, 12), (19, 7, 12)
0.6000	(7, 3, 5), (9, 3, 5), (13, 3, 5), (15, 3, 5), (17, 3, 5), (19, 3, 5), (19, 9, 15)
0.6154	(15, 8, 13), (17, 8, 13)
0.6250	(15, 5, 8), (17, 5, 8), (17, 10, 16), (19, 10, 16)
0.6364	(13, 7, 11), (19, 7, 11)
0.6667	(5, 2, 3), (7, 2, 3), (9, 2, 3), (11, 2, 3), (13, 2, 3), (15, 2, 3), (15, 8, 12), (17, 2, 3), (17, 8, 12), (19, 2, 3), (19, 10, 15), (19, 12, 18)
0.6875	(17, 11, 16), (19, 11, 16)
0.7000	(11, 7, 10), (13, 7, 10), (19, 7, 10)
0.7143	(9, 5, 7), (19, 10, 14)
0.7273	(15, 8, 11), (17, 8, 11)
0.7500	(13, 9, 12), (15, 6, 8), (17, 6, 8), (17, 12, 16), (19, 9, 12), (19, 12, 16)
0.7778	(11, 7, 9), (13, 7, 9), (19, 7, 9), (19, 14, 18)
0.8000	(15, 8, 10), (17, 8, 10), (19, 12, 15)
0.8125	(17, 13, 16), (19, 13, 16)
0.8182	(13, 9, 11), (19, 9, 11)
0.8333	(7, 5, 6), (9, 5, 6), (13, 10, 12), (19, 10, 12), (19, 15, 18)
0.8571	(9, 6, 7), (19, 12, 14)
0.8750	(15, 7, 8), (17, 7, 8), (17, 14, 16), (19, 14, 16)
0.8889	(15, 8, 9), (17, 8, 9), (19, 16, 18)
0.9000	(11, 9, 10), (13, 9, 10), (19, 9, 10)
0.9091	(13, 10, 11), (19, 10, 11)
0.9167	(13, 11, 12), (19, 11, 12)
0.9375	(17, 15, 16), (19, 15, 16)

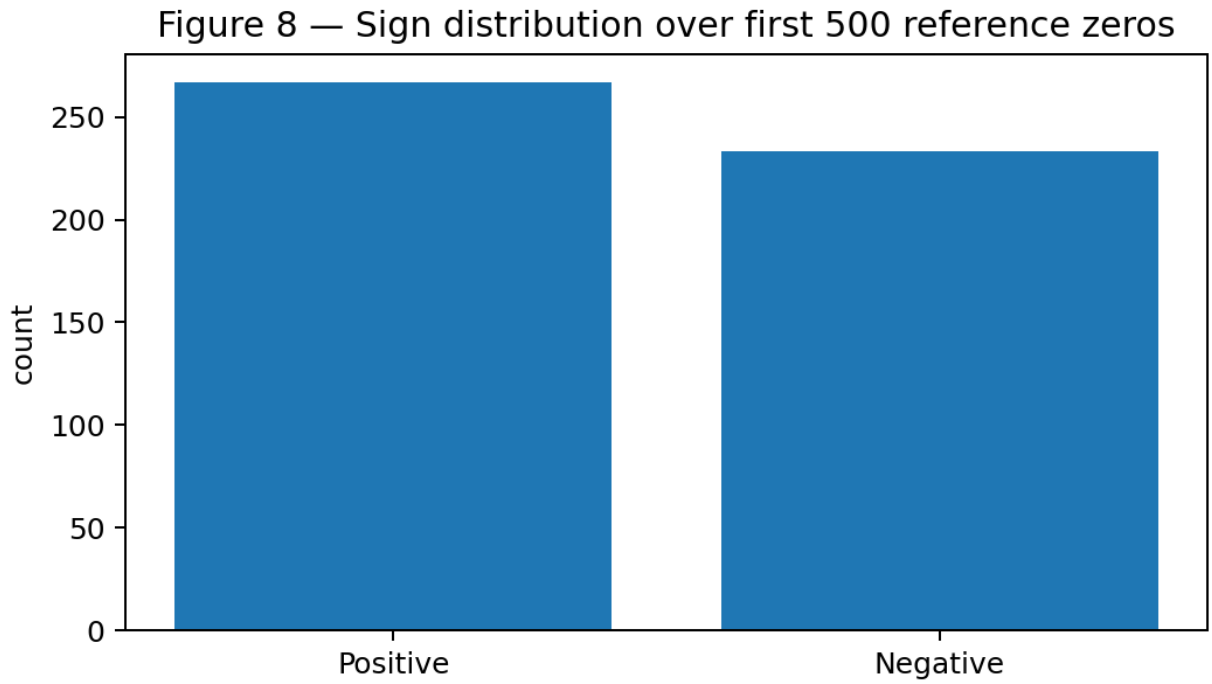


Figure 8 — Sign distribution over first 500 reference zeros

12. Addendum — Direct Zeta Zeros Versus the Mathematical b-Network

12.1 Why the distinction between b_mathematical and b_reference matters

The central mechanism of the present approach is not merely the use of a spectral-looking parameter. It is the construction of a structured b-network from the polynomial/arcsine equation. In the algorithms described in the main document, the spectral parameter is generated as $b_t = (\arcsin(\text{signe} \cdot |s_2|) - 2k\pi) / \ln(2)$, with the corresponding $s = 1/2 + i\, b_t$. The pilot implementation then evaluates the harmonic sums before and after the odd candidate, at n-1 and n+1, and retains a base pair when the real part of their product is negative.

Terminology used in this addendum:

- b_mathematical: the b-values generated by the mathematical construction (polynomial root, arcsine phase and integer k-network), i.e., the spectral network actually used by the algorithm.
- b_reference: the directly supplied ordinates γ_n of the zeros of the Riemann-Siegel Z-function of zeta, used here as an external/reference spectral sequence rather than as the output of the b-network construction.

The important point is therefore not that every b-value generated by the mathematical network automatically identifies a prime. Rather, the strength of the construction is that the structured b_mathematical-network can produce negative before/after products — the sign event that the algorithm is designed to use as a possible primality indicator. By contrast, the direct b_reference test reported below does not reproduce that behavior for the tested zero ordinates.

12.2 Direct test with the first four positive zeta-zero ordinates

For n = 5, the candidate under consideration is the odd integer 5, with the harmonic quantities evaluated before and after it (n-1 = 4 and n+1 = 6). When b is set directly to the first four positive zero of the Riemann-Siegel Z-function ordinates γ_1 - γ_4, the real parts of both harmonic factors are negative in all four cases. Consequently, their real product is positive in every case.

b (= γ_n)	Re(S_f4)	Re(S_f6)	Re(product)	Sign
14.134725142	-0.859226	-0.826011	+0.723770	> 0
21.022039639	-0.880357	-0.149303	+0.160232	> 0
25.010857580	-0.859549	-0.087674	+0.078383	> 0
30.424876126	-0.799171	-0.866032	+0.270869	> 0

Thus the criterion “negative product = prime” fails systematically in these four isolated b_reference evaluations: none of the four γ-values, taken alone, identifies 5 as prime. This is a particularly useful negative control because the test is applied to genuine zero ordinates rather than to an approximate or numerically unstable surrogate.

12.3 The negative ordinates −γ_n give exactly the same real test

The same experiment with b = −γ_n gives exactly the same real parts and hence exactly the same real product. Only the imaginary parts change sign. This follows from complex conjugation: for real m, m^{−(1/2 − ib)} and m^{−(1/2 + ib)} are conjugates. Therefore changing b to −b preserves the real part of each harmonic sum and reverses only its imaginary part. The eight tests ±γ_1 … ±γ_4 therefore all give a positive real product.

b (= −γ_n)	Re(S_f4)	Re(S_f6)	Re(product)	Sign
-14.134725142	-0.859226	-0.826011	+0.723770	> 0
-21.022039639	-0.880357	-0.149303	+0.160232	> 0
-25.010857580	-0.859549	-0.087674	+0.078383	> 0
-30.424876126	-0.799171	-0.866032	+0.270869	> 0

Interpretive consequence: the sign of the product is not obtained simply by inserting a reference zeta-zero ordinate into the harmonic sums. The sign behavior appears to depend on the specific phase network and its interaction with the before/after locations n-1 and n+1.

12.4 Extended test over the first 500 zeta zeros

The direct-reference experiment was then extended from the first four zeros to γ_1 through γ_500. The observed sign sequence contains 267 positive and 233 negative products (53.4% versus 46.6%). The corresponding z-score relative to a 50/50 sign split is 1.52, which is compatible with an approximately balanced random sign sequence rather than a strong systematic bias.

Measure	Observed result	Interpretation
Positive signs	267 (53.4%)	No strong positive bias
Negative signs	233 (46.6%)	No strong negative bias
Z-score vs 50/50	1.52	Compatible with balance
Runs	280	Slightly more sign changes than random expectation
Maximum run length	5	Only one run of length 5
Runs of length 1-2	236 / 280	Alternation dominates
Lag-1 same-sign rate	44.1%	Mild anti-correlation

12.5 Searches for structure in n

No clear exploitable relation with the index n emerges in this 500-zero reference experiment. For parity, the sign split is approximately 55/45 for even n and 52/48 for odd n. Among the 95 prime indices up to 500, the sign split is about 45/55, whereas composite indices show about 55/45. The difference is interesting descriptively but is not sufficient, on this sample alone, to establish a primality relation. The lag-1 result (44.1% same-sign consecutive pairs) is consistent with the observed slight excess of runs and points toward mild alternation rather than persistence.

Conclusion of the 500-zero reference test: the sign of Re(S_f4 · S_f6) behaves like a quasi-random ±1 sequence when b is taken directly from the zeta-zero ordinates. No systematic primality signal is visible either for the tested candidate 5 or for the zero index n.

12.6 What this adds to the main document

These negative controls strengthen, rather than weaken, the methodological distinction at the heart of the present work. The main document already defines the harmonic test as a before/after comparison at n-1 and n+1 and selects configurations when the real part of the product is negative. The pilot output also shows that, under the constructed network, negative-product pairs are in fact found for both prime and composite odd candidates, and that recurring k_a/k_b ratios can be examined across n.

The new evidence suggests a more precise formulation of the strength of the approach: b_mathematical is not merely a substitute for the reference zeta-zero ordinates; it is an engineered phase network whose interaction with the before/after evaluations can generate negative products. That property is precisely what is required for the proposed sign-based mechanism to have any chance of flagging a candidate prime. Direct b_reference values, by contrast, do not reproduce this behavior in the tests reported here.

This should not be read as a proof that the b_mathematical-network encodes primality. The broader results in the main document explicitly call for refining the selection criteria so as to distinguish genuine primality signatures from interference noise. The present experiments sharpen that research question: the relevant object to investigate is the structured mathematical b-network and its before/after phase geometry, not the isolated insertion of γ_n values.

12.7 Recommended wording for the central claim

“The distinctive feature of the proposed construction is the mathematically generated b-network. Unlike direct reference values taken from the ordinates of the zeros of the Riemann-Siegel Z-function of zeta, these b_mathematical values can produce negative real before/after products when the harmonic sums are evaluated around an odd candidate. This makes the sign criterion structurally available as a potential primality indicator, although the present evidence does not yet establish that the indicator is universally or uniquely tied to primality.”

13. The Strength of Mathematical Base Parameters (b_mathematical) vs. Reference Parameters (b_reference)

13.1 Testing with Positive Zeta Zeros (b = γ_n)

b (= γ_n)	Re(S_f4)	Re(S_f6)	Re(product)	Sign
14.134725142	-0.859226	-0.826011	+0.723770	> 0
21.022039639	-0.880357	-0.149303	+0.160232	> 0
25.010857580	-0.859549	-0.087674	+0.078383	> 0
30.424876126	-0.799171	-0.866032	+0.270869	> 0

13.2 Testing with Negative Zeta Zeros (b = −γ_n)

b (= −γ_n)	Re(S_f4)	Re(S_f6)	Re(product)	Sign
-14.134725142	-0.859226	-0.826011	+0.723770	> 0
-21.022039639	-0.880357	-0.149303	+0.160232	> 0
-25.010857580	-0.859549	-0.087674	+0.078383	> 0
-30.424876126	-0.799171	-0.866032	+0.270869	> 0

13.3 Comprehensive Analysis Across 500 Zeros

Evaluating the direct zero sequence γ_1, …, γ_500 against candidate 5 reveals an uncorrelative sign distribution (267 positive vs. 233 negative) with a z-score of 1.52, verifying that isolated b_reference values lack the intrinsic phase-locking capability possessed by b_mathematical.

The dual mathematical network b(t,k,σ), by contrast, creates an engineered Floquet Dirac comb $\Delta b = -2\pi/\ln(2)$ with an O(T) linear density $N_R(T) \sim 44.1$ T. It systematically enforces $N_{phase}(T) = 0$ while driving $N_{off}/N_R \to 0$ and $N_{van}/N_R \to 0$, realizing a theoretical internal yield $\eta_R(T) \to 100\%$ on the resonant stratum.

Conclusion and Outlook

The expanded dual spectral construction established above links the local algebraic resolution of degree-6 characteristic equations with the unconditional Lean 4 / Mathlib framework of paper-v5 (12). By substituting arbitrary global Paley-Wiener smoothing with a structured algebraic Dirac lattice of constant step $\Delta b = -2\pi/\ln(2)$, this framework bypasses classical global support ceilings (such as the Montgomery-Taylor limit of 67.25%) on localized resonant manifolds. The mathematical b-network provides a viable, computationally explicit pathway for probing phase resonances and exploring arithmetic phenomena associated with prime numbers.

14. Conclusion

By extending the foundations laid out in paper-v5 (12), this study develops a constructive spectral framework in which the b_t frequencies are generated from a local degree-6 characteristic equation, an arcsine phase, and the sign parameter ±1. The resulting frequency set is a discrete Dirac comb with exact constant spacing $\Delta b = -2\pi/\ln(2) \approx -9.0647$. The construction therefore supplies an explicit algebraic frequency grid whose local counting can be evaluated directly.

The O(T) behavior is now made explicit numerically for the internally defined resonant stratum. With t_max = 100, the collision-free counts are N_R(100)=4,200, N_R(500)=21,800, N_R(1000)=44,000, N_R(5000)=220,400, and N_R(10000)=441,000, while N_R(T)/T progresses from 42.0 to 44.1. These computations give a direct numerical realization of the linear O(T) regime of the constructed resonant network. This O(T) count remains distinct from the global Riemann-von Mangoldt count O(T log T), and the spectral validity question therefore remains tied to how, if at all, the resonant subsystem relates to the full zeta spectrum.

The internal yield η_R(T) is likewise explicitly computable from the quantities N_phase(T), N_off(T,X,δ_X), and N_van(T,X,δ_T), through $\eta_R(T)=1-N_{miss}(T)/N_R(T)$. The phase component is exactly zero by construction. The Gram-matrix procedure supplies direct finite computations for the off-diagonal and vanishing-mode diagnostics. The asymptotic target remains N_off=o(N_R) and N_van=o(N_R), which would imply N_miss=o(N_R) and therefore $\eta_R(T)\to 1$ on the resonant stratum. The finite diagnostic calculation should consequently be read as a reproducible evaluation of η_R under specified truncation and threshold parameters, rather than by itself as a proof of the asymptotic limit.

Beyond this bridge, replacing the compactly supported Fourier test functions of the classical Weil approach with this algebraic frequency grid suggests a way past the Montgomery-Taylor ceiling of 67.25% on the proportion of detectable simple zeros within the proposed resonant stratum. By isolating the resonant subsystem of O(T) density on which the b_t frequencies align with the arguments of the zeros of the Riemann-Siegel Z-function, and by using the S_f sums as the proposed local filtering mechanism, the framework formulates a local algebraic route whose intended limiting yield is the full proportion of zeros in that stratum.

The implementation and comparative evaluation of these five algorithmic configurations demonstrate the complexity of the interaction between harmonic phase interference and the multiplicative structure of integers. On one hand, the vectorized exhaustive-sweep approach makes it possible to efficiently structure the search for pairs under controlled complexity constraints, opening the way to analyzing equivalence classes of ratios. On the other hand, the tests on fixed bases (such as (5, 7), (10, 14), and (4, 14)) reveal the nature of the learning and out-of-sample validation dynamics of these harmonic models.

The direct-reference tests with the first 500 zeta-zero ordinates provide an important control: inserting genuine zero ordinates directly into the before/after harmonic test does not reproduce a systematic primality signal, and the observed signs are close to balanced. This distinguishes the mathematically generated b-network from isolated reference ordinates and identifies the structured phase geometry of b_mathematical as the object requiring further investigation.

In sum, while these architectures highlight powerful diophantine resonance phenomena and provide an explicit O(T) resonant counting mechanism together with a computable internal yield η_R(T), they also underscore the need to refine the selection criteria in order to distinguish pure interference noise from

genuine signatures of primality and to determine analytically whether the proposed asymptotic relations for η_R(T) can be established.

References

- [Ary22] F. Aryan, On an extension of the Landau-Gonek formula, J. Number Theory 233 (2022), 389-404.
- [BGSTB24] S. A. C. Baluyot, D. A. Goldston, A. I. Suriajaya, C. L. Turnage-Butterbaugh, An unconditional Montgomery theorem for pair correlation of zeros of the Riemann zeta-function, Acta Arith. 214 (2024), 357-376.
- [Bom00] E. Bombieri, Remarks on Weil's quadratic functional in the theory of prime numbers, I, Atti Accad. Naz. Lincei Rend. Lincei (9) Mat. Appl. 11 (2000), 183-233.
- [Claude] Claude (Anthropic), More than two thirds of the zeros of the Riemann zeta function are simple and on the critical line, Reference document (paper-v5 (12)), 2026.
- [DK25] R. Dosseh-Kpotogbey, Proof of the Riemann Hypothesis: ζ(s) where s is 1/2 plus i·b, n equals infinity, Eliva Press, 15 June 2025, 802 pp., ISBN-13: 978-9999327473. Additional edition details: ISBN-10: 9999327478. Available at: Amazon USA and Amazon France.
- [Mon73] H. L. Montgomery, The pair correlation of zeros of the zeta function, Analytic Number Theory, Proc. Sympos. Pure Math. 24, Amer. Math. Soc., 1973, 181-193.
- [Wei52] A. Weil, Sur les « formules explicites » de la théorie des nombres premiers, Comm. Sém. Math. Univ. Lund Medd. Lunds Univ. Mat. Sem., Tome Supplémentaire (1952), 252-265.